

Chapter 13

Contents

Chapters

13. Programming CCP & ECCP & PWM

13.1 Overview of CCP

13.2 CCP Modes of Operation

13.2.1 Compare

13.2.2 Capture

13.2.3 PWM

13.3 Using ECCP

13.1 Overview of CCP

CCP stands for Capture, Compare & PWM module. A timer is helpful for various applications, and it alone is not sufficient for certain applications. Some applications that include determining the frequency of an input signal, finding the width of the pulse, generate a waveform with a required pulse width etc. need additional hardware in addition to timers. CCP module is one such additional hardware included in PIC microcontrollers that could be used for such applications. Each CCP module contains a 16-Bit CCPxCON register to control the CCP module, a 16-Bit holding register combination i.e CCPRxL (Low byte) & CCPRxH (High byte) to hold the capture, compare or PWM value. These registers are readable and writable. The details of the registers are presented in the following sections corresponding to the operating modes. The number of CCP modules present in some of the PIC microcontrollers is presented in the table 13.1.

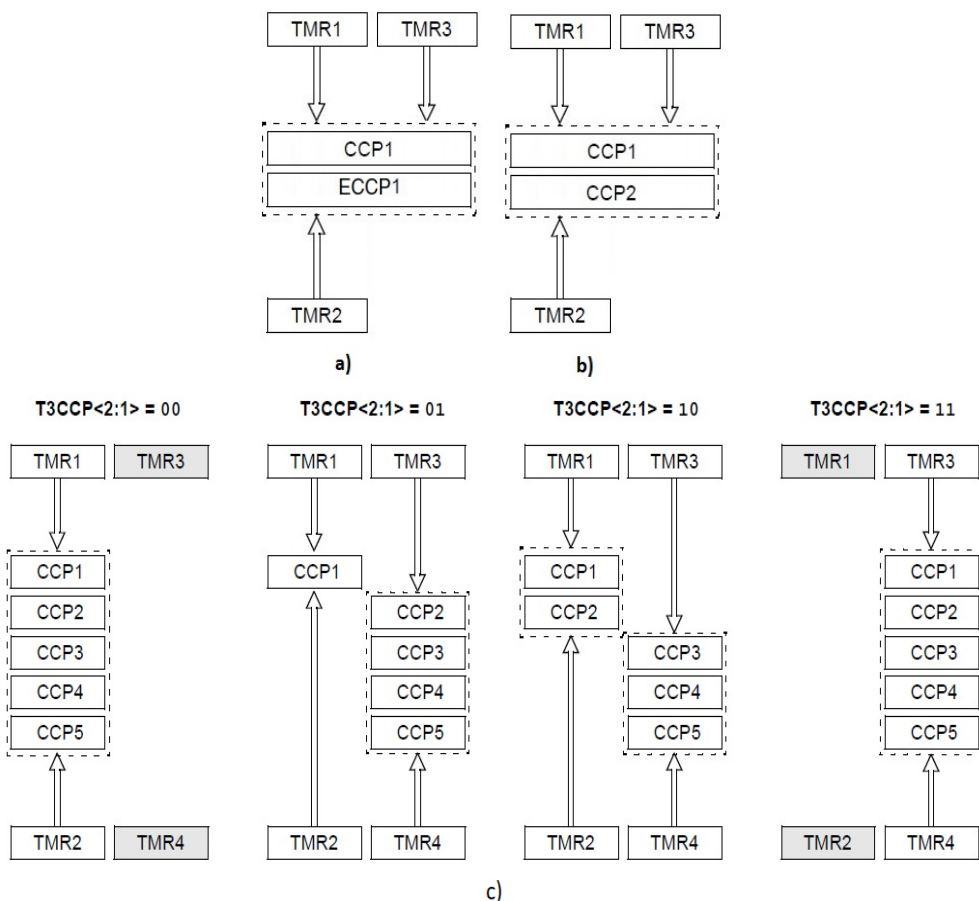


Fig 13.1 CCP and Timer interconnect configurations:
a) PIC18F458 b) PIC18C801 c) PIC18F8720

Table 13.1: CCP modules & Registers in PIC Devices

Device	No of CCP's	Module Name	Registers	No of ECCP's	Module Name	Registers
PIC18F458	1	CCP1	CCP1CON CCPR1H CCPR1L	1	ECCP1	ECCP1CON ECCPR1H ECCPR1L ECCPAS ECCP1DEL
PIC18C801	2	CCP1	CCP1CON CCPR1H CCPR1L			
		CCP2	CCP2CON CCPR2H CCPR2L			
PIC18F8720	5	CCP1	CCP1CON CCPR1H CCPR1L			
		CCP2	CCP2CON CCPR2H CCPR2L			
		CCP3	CCP3CON CCPR3H CCPR3L			
		CCP4	CCP4CON CCPR4H CCPR4L			
		CCP5	CCP5CON CCPR5H CCPR5L			
		CCP5	CCP5CON CCPR5H CCPR5L			

Table 13.2: CCP1 Timer Sources

CCP1/ECCP1 Mode	Timer Resource
Capture	Timer1 or Timer3
Compare	Timer1 or Timer3
PWM	Timer2 (or Timer 4 – only in PIC18F8720)

Table 13.3a): CCP Pins for PIC18F458 & PIC18C601/801 devices

PIC18F458		PIC18C601/801	
CCP Module	Pin No	CCP Module	Pin No
CCP1	RC2	CCP1	RC2
ECCP1	RD4	CCP2	RB3

Table 13.3b): CCP Pins for PIC18F8720

PIC18F8720		
CCP Module	Pin No	Comments
CCP1	RC2	
CCP2	RC1	RC1 is default assignment when CCP2MX configuration bit is set in CONFIG3H.
	RE7	CCP2 is multiplexed with RE7 pin by default when the device is configured in Microcontroller mode. CCP2 is multiplexed with RB3 or RC1 in other modes. Alternate assignment for CCP2 when CCP2MX configuration bit is cleared (not selected) in CONFIG3H.
	RB3	RB3 is alternate assignment for CCP2 when CCP2MX configuration bit is cleared (not selected) in CONFIG3H when the device is operating in Microprocessor, Microprocessor with Boot Block or Extended Microcontroller mode.
		CCP2MX: CCP2 Mux bit In Microcontroller mode: 1 = CCP2 input/output is multiplexed with RC1 0 = CCP2 input/output is multiplexed with RE7 In Microprocessor, Microprocessor with Boot Block and Extended Microcontroller modes: 1 = CCP2 input/output is multiplexed with RC1 0 = CCP2 input/output is multiplexed with RB3
CCP3	RG0	
CCP4	RG3	
CCP5	RG4	

CONFIG3H: Configuration Register 3 HIGH Register Format

U-0	U-0	U-0	U-0	U-0	U-0	R/P-1	R/P-1
—	—	—	—	—	—	r	CCP2MX
bit 7						bit 0	

Bit No	Bit	Description
bit 7-2	Unimplemented	Read as '0'
bit 1	Reserved	Read as unknown
bit 0	CCP2MX	CCP2 Mux bit <u>In Microcontroller mode:</u> 1 = CCP2 input/output is multiplexed with RC1 0 = CCP2 input/output is multiplexed with RE7 <u>In Microprocessor, Microprocessor with Boot Block and Extended Microcontroller modes:</u> 1 = CCP2 input/output is multiplexed with RC1 0 = CCP2 input/output is multiplexed with RB3

13.2 CCP Modes of Operation

In PIC microcontrollers, the CCP module in conjunction with timers could be configured to operate in a specific mode based on the application. The CCP module could be operating in any one of the following 3 modes.

- Capture Mode
- Compare Mode
- PWM Mode

Registers related to Capture, Compare, PWM, Timer1, Timer 2, Timer3 & Timer4

Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
CCPRxL (1)	Capture/Compare/PWM Register x (LSB)							
CCPRxH (1)	Capture/Compare/PWM Register x (MSB)							
CCPxCON (1)	—	—	DCxB1	DCxB0	CCPx M3	CCPx M2	CCPx M1	CCPx M0
TRISC	PORTC Data Direction Register for CCP1							
TMR1L	Holding Register for the Least Significant Byte of the 16-bit TMR1 Register							
TMR1H	Holding Register for the Most Significant Byte of the 16-bit TMR1 Register							
T1CON	RD16	—	T1CK PS1	T1CK PS0	T1OSC EN	T1SYNC	TMR1 CS	TMR1 ON
TMR3H	Timer3 Register High Byte							
TMR3L	Timer3 Register Low Byte							
T3CON	RD16	T3CCP 2	T3CK PS1	T3CK PS0	T3CCP 1	T3SYNC	TMR3 CS	TMR3 ON
INTCON	GIE/ GIEH	PEIE/ GIEL	TMR0IE	INT0IE	RBIE	TMR0IF	INT0IF	RBIF
PIR1	PSPIF(3)	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF
PIE1	PSPIE(3)	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE
IPR1	PSPIP(3)	ADIP	RCIP	TXIP	SSPIP	CCP1IP	TMR2IP	TMR1IP
PIR2	—	CMIF(3)	—	EEIF(3)	BCLIF	LVDIF	TMR3IF	CCP2IF(2)
PIE2	—	CMIE(3)	—	EEIE(3)	BCLIE	LVDIE	TMR3IE	CCP2IE(2)
IPR2	—	CMIP(3)	—	EEIP(3)	BCLIP	LVDIP	TMR3IP	CCP2IP(2)
RCON	IPEN	—	—	<u>RI</u>	<u>TO</u>	<u>PD</u>	<u>POR</u>	<u>BOR</u>

On PIC18F458

PIR3, PIE3, IPR3 are present but are not used for CCP1. They are not present in PIC18C601/801

On PIC18C601/801

TRISB PORTB Data Direction Register used for CCP2

Only on PIC18F8720

Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
TRISB	PORTB Data Direction Register. Used in other Modes with CCP2MX = 0.							
TRISE	PORTE Data Direction Register. Used in Microcontroller Mode with CCP2MX = 0							
TRISG	PORTG Data Direction Register. Used for CCP3, CCP4 and CCP5							
PIR3	—	—	RC2IF	TX2IF	TMR4IF	CCP5IF	CCP4IF	CCP3IF
PIE3	—	—	RC2IE	TX2IE	TMR4IE	CCP5IE	CCP4IE	CCP3IE

IPR3	—	—	RC2IP	TX2IP	TMR4IP	CCP5IP	CCP4IP	CCP3IP
PWM Specific Registers for all devices								
Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
TMR2	Timer2 Module Register							
PR2	Timer2 Module Period Register							
T2CON	—	T2OUT PS3	T2OUT PS2	T2OUT PS1	T2OUT PS0	TMR2 ON	T2CK PS1	T2CK PS0
PWM Specific Registers Only on PIC18F8720								
Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
TMR4	Timer4 Register							
PR4	Timer4 Period Register							
T4CON	—	T4OUT PS3	T4OUT PS2	T4OUT PS1	T4OUT PS0	TMR4 ON	T4CK PS1	T4CK PS0

Note 1: 'x' identifies the individual module (CCP1 through CCP5).

Note 2: CCP2 not in PIC18F458, Used for ECCP1. Note 3: Not present in PIC18C601/801.

CCPxCON: Capture Compare x Control Register Format

U-0	U-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
—	—	DCxB1	DCxB0	CCPxM3	CCPxM2	CCPxM1	CCPxM0

bit 7

bit 0

Bit No	Bit	Description
bit 7-6	Unimplemented	Read as '0'
bit 5-4	DCxB1:DCxB0	PWM Duty Cycle bit 1 and bit 0 for CCP Module x <u>Capture mode:</u> Unused. <u>Compare mode:</u> Unused. <u>PWM mode:</u> These bits are the two LSBs (bit 1 and bit 0) of the 10-bit PWM duty cycle. The upper eight bits (DCx9:DCx2) of the duty cycle are found in CCPxL.
bit 3-0	CCPxM3:CCPxM0	CCP Module x Mode Select bits 0000 = Capture/Compare/PWM off (resets CCPx module) 0001 = Reserved 0010 = Compare mode, toggle output on match (CCPxIF bit is set) 0011 = Capture mode, CAN message received (CCP1 only) In PIC18F458 Reserved in PIC18C601/801, PIC18F8720 0100 = Capture mode, every falling edge 0101 = Capture mode, every rising edge 0110 = Capture mode, every 4th rising edge 0111 = Capture mode, every 16th rising edge 1000 = Compare mode, initialize CCP pin low, on compare match force CCP pin high (CCPxIF bit is set) 1001 = Compare mode, initialize CCP pin high,

on compare match force CCP pin low
(CCPIF bit is set)
1010 = Compare mode, Generate software interrupt on
compare match
(CCPIF bit is set, CCP pin is unaffected)
1011 = Compare mode, trigger special event
(CCPIF bit is set) See details in trigger spl event.
11xx = PWM mode

13.2.1 Capture Mode

The CCP module operating in capture mode makes use of CCPxCON register to control the CCP module and 16-Bit CCPRxL (Low byte) & CCPRxH (High byte) register to hold the captured time value, i.e. the time (16-bit value of TMR1 (TMR1H & TMR1L) or TMR3 (TMR3H & TMR3L) register at which the event occurred. The CCPxIF flag bit of the respective CCPx module is set when the event is detected. The type of events that are considered in the capture mode are:

- Every falling edge
- Every rising edge
- Every 4th rising edge
- Every 16th rising edge

The CCPxCON<3:0> bits i.e. CCPxM3:CCPxM0 are used to select kind of event and the prescale value that needs to be captured. An interrupt could be generated when the CCPxIF flag is set if the corresponding interrupts are enabled (i.e. enable bits are set). The CCPxIF flag needs to be cleared in software and the captured event time value in CCPRxH & CCPRxL needs to be read and stored separately. Otherwise, earlier captured value is overwritten.

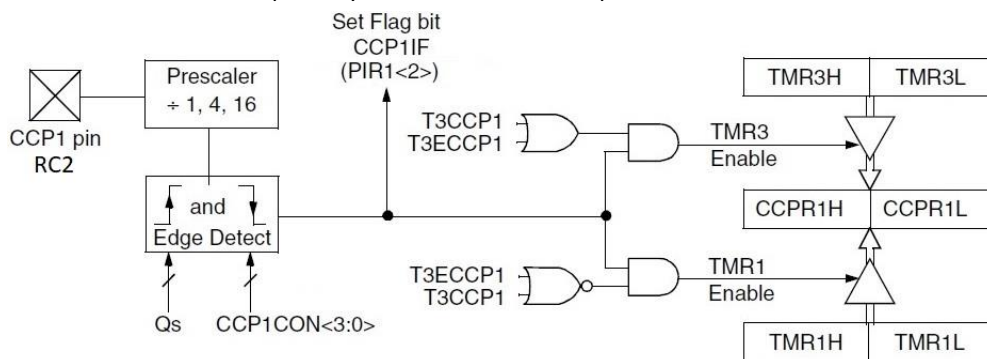


Fig 13.2 a) Block Diagram of Capture Mode: CCP1 in PIC18F458

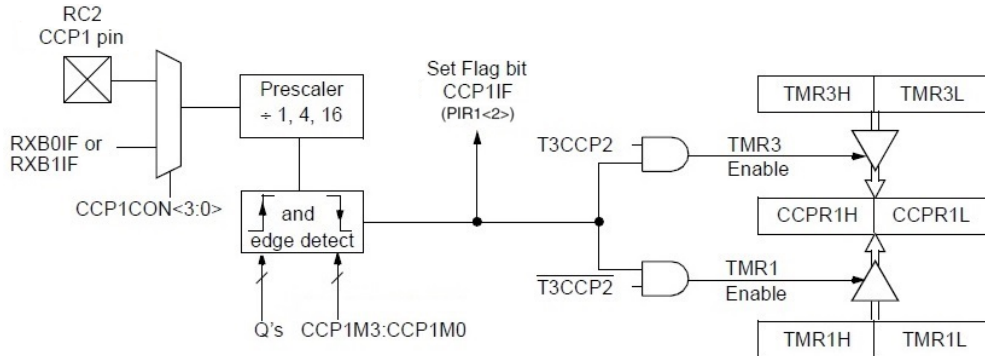


Fig 13.2 b) Block Diagram of Capture Mode: CCP1 in PIC18C601/801

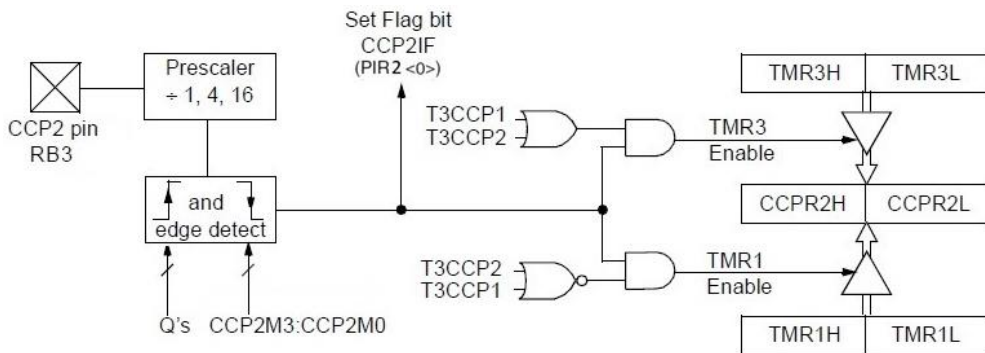
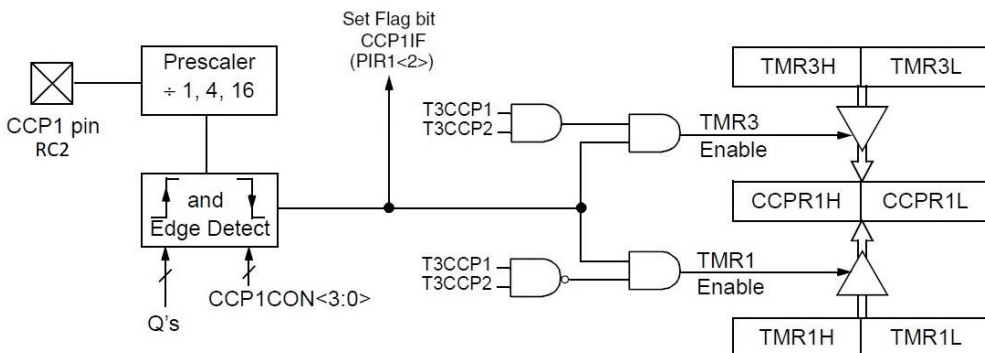


Fig 13.2 c) Block Diagram of Capture Mode: CCP2 in PIC18C601/801



Note: CCP2 pin is configurable on RC1/RE7/RB3
CCP3, CCP4 and CCP5 are available on RG0, RG3 and RG4 respectively.

Fig 13.2 d) Block Diagram of Capture Mode: CCP1 in PIC18F8720

The CCPx pin need to be configured as input by setting the appropriate TRIS bit. If the CCPx pin is in output mode, the write to port may cause a capture event.

For working of capture mode, the timer1 and/or timer 3 need to be running in timer mode or synchronized counter mode. The T3CON<6,3> bits T3CCP2 :T3CCP1 bits of timer 3 control register are used to select which timer is used for the respective CCP module.

Whenever a capture mode need to be changed, the CCPxIE and CCPxIF bits need to be cleared so that a false capture interrupt is not generated. Also a change of prescaler value may generate a non prescale input as the prescale counter is not changed. Hence to clear the prescale counter and to ensure generation of interrupt based on actual prescale the CCP module need to be switched off. When the CCP module is switched off the prescale counter is cleared. The recommended procedure to change value of prescaler is to clear the CCPxCON register and then load the appropriate CCPx prescaler settings.

The capture modes of CCP2, CCP3, CCP4 & CCP5 of PIC18F8720 uses equivalent structures based on the CCP & Timer interconnect configurations based on T3CCP<2:1> bits of T3CON register discussed earlier.

The CCP is configured in capture mode in applications where time period or frequency need to be determined by detecting the time difference between the two same type of edges. Also, the capture mode is used for determining the pulse width by detecting the time difference between two different type of edges.

Configuring/Setup examples for Capture Operation

1. Initialize the CCPxCON register to operate in Capture mode with respective prescale. i.e CCPxCON<3:0> bits i.e CCPxM3:CCPxM0. (Ex: Set of rising edge)
2. Configure the respective CCPx pin as input pin by setting the respective TRIS bit.
3. Initialize Timer 3 control register T3CON<6,3> bits T3CCP2 :T3CCP1 bits to select Timer 1 or Timer 3 as source for the CCPx module.
4. Based on the selected timer Initialize the control register of Timer 1 (or Timer 3) with its prescale.
5. Initialize the Timer 1 register - (TMR1H & TMR1L) or Timer 3 (TMR3H & TMR3L) registers to appropriate value – typically zero.
6. Clear the Timer1 or Timer 3 and CCPxIF flags
7. If required, Enable the CCPxIF interrupt and the required global interrupt.

Measuring the Time Period (When the frequency of captured signal is less than timers)

Method 1: Measuring the Time Period/Frequency – Demonstrated in Program 13.1

8. Keep checking for CCPxIF for the first event (ex: rising edge),
 - if CCPxIF is set, Clear the CCPxIF flag
9. Start/Enable the Timer1 (or Timer3)

10. Keep checking for CCPxIF for the second event (same type as first event – if rising edge is considered for first, it is to be rising edge for second also),
 - if CCPxIF is set, Clear the CCPxIF flag
 11. Stop/Disable the Timer1 (or Timer3)
 12. Copy the second event captured timer value that's present in CCPRxH & CCPRxL to a location m and determine the time period/frequency based on the prescale.
- Note:** Instead of considering first and second event, second and third event may be considered for accuracy because that is what is generally repetitive.

Method 2: Measuring the Time Period/Frequency & get initial delay - Demonstrated in Program 13.2. Such a case is also required when the timer is used for some other purposes.

8. Start/Enable the Timer1 (or Timer3). (Optional: May have other application logic based on timer)
9. Keep checking for CCPxIF for the first event (ex: rising edge),
 - if CCPxIF is set copy the first event captured timer value that's present in CCPRxH & CCPRxL to a location n. This value gives the initial delay.
 - Clear the CCPxIF flag
10. Keep checking for CCPxIF for the second event (same type as first event – if rising edge is considered for first, it is to be rising edge for second also),
 - if CCPxIF is set, copy the second event captured timer value that's present in CCPRxH & CCPRxL to a location m.
 - Clear the CCPxIF flag
11. Stop/Disable the Timer1 (or Timer3)
12. Subtract the content of n from m. i.e. m-n to determine the number timer pulses between the two events and determine the time period/frequency based on the prescale.

Note: When frequency of captured signal is more than timers, the number of captured events can be counted in a fixed timer period to determine the captured events time period/frequency. This can be achieved by just using the timer with input signal at T1CKI pin for rising edge detection, discussed as an example for counter application. If a falling edge detection is required, then CCP based capture is required. The timer can be sourced from main oscillator or from T1CKI pin.

Measuring the Pulse width

Demonstrated in Program 13.5

8. Keep checking for CCPxIF for the first event (ex: rising edge),
 - if CCPxIF is set, Clear the CCPxIF flag

- Clear LSB bit of CCPxCON such that CCPxCON<3:0> bits i.e. CCPxM3:CCPxM0 are such that the edge event is opposite of current state (to make falling edge).
9. Start/Enable the Timer1 (or Timer3)
 10. Keep checking for CCPxIF for the second event (different event type as first event – if rising edge is considered for first, it is to be falling edge for second),
 - if CCPxIF is set, Clear the CCPxIF flag
 - Set LSB bit of CCPxCON such that CCPxCON<3:0> bits i.e. CCPxM3:CCPxM0 are such that the edge event is opposite of current state (to make rising edge).
 11. Stop/Disable the Timer1 (or Timer3)
 12. Copy the second event captured timer value that's present in CCPRxH & CCPRxL to a location m and determine the pulse width based on the prescale.
- Method 2 equivalent of Measuring the Time Period/Frequency may be used for pulse width.

Program 13.1: Determining the Time Period/Frequency, Store No of pulses in 0x30,0x31 and send to PortB and PortD. Using CCP1 and Timer 1 in capture mode. The logic is demonstrated in Fig P13.1

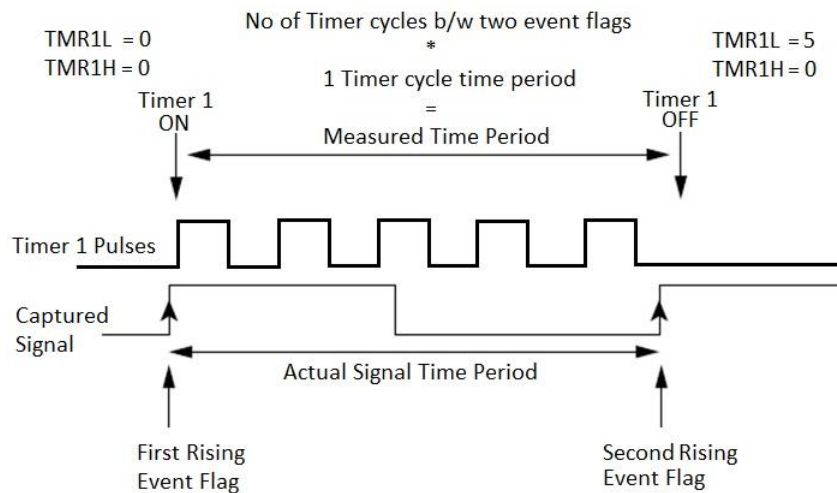


Fig P13.1: Logic demonstration of program 13.1.

ORG	0x0000	; Set Origin 0x0000 of Flash memory
GOTO	START	; Go to Start of Main program.
ORG	0x0030	; Set Origin 0x0030 for Main program
START	CLRF	TRISB ; Make Port B as output
	CLRF	TRISD ; Make Port D as output

	MOVLW	0X05	; WREG = 0x05
	MOVWF	CCP1CON	; CCP1CON = WREG = 0x05, Setting CCP1 ; for Capture mode with rising edge with 0 ; prescale
	BSF	TRISC,2	; Make CCP1/RC2 pin as input for capture
	MOVLW	0x00	; WREG = 0x00
	MOVWF	T3CON	; T3CON = 0x00, Cleared to Select Timer 1 ; as time source for CCP
	MOVWF	T1CON	; T1CON =0x00, Select 8-bit read/write ; operations of Timer1, with Prescale 1, ; select internal main clock fosc/4
UP	CLRF	TMR1H	; Initialize the Timer 1 high count register ; TMR1H to 0x00
	CLRF	TMR1L	; Initialize the Timer 1 low count register ; TMR1L to 0x00
	BCF	PIR1,CCP1IF	; Clear CCP1IF flag
L1	BTFSS	PIR1,CCP1IF	; Wait for first rising edge event. Check ; CCP1IF flag. Skip next instruction if event ; flag
	BRA	L1	; Branch to L1 if event flag not set
	BSF	T1CON,0 ;TMR1ON	; Start the Timer 1 to see number of pulses ; till next event flag.
	BCF	PIR1,CCP1IF	; Clear CCP1IF flag
L2	BTFSS	PIR1,CCP1IF	; Wait for second rising edge event. Check ; CCP1IF flag. Skip next instruction if event ; flag
	BRA	L2	; Branch to L2 if event flag not set
	BCF	T1CON,0 ;TMR1ON	; Stop the Timer 1.
	MOVF	CCPR1H,W	; WREG = CCPR1H, Copy the latched Timer1 ; High
	MOVWF	0x31	; [0x31] = CCPR1H = WREG
	MOVF	CCPR1L,W	; WREG = CCPR1L, Copy the latched Timer1 ; Low
	MOVWF	0x30	; [0x30] = CCPR1L = WREG
	MOVFF	0x30,PORTB	; Send Low pulse count to Port B, PORTB = ; CCPR1L = [0x30]
	MOVFF	0x31,PORTD	; Send High pulse count to Port D, PORTD = ; CCPR1H = [0x31]
	GOTO	UP	; Goto UP, Repeat this process ; continuously.
	END		

Program 13.2: Determining the Time Period/Frequency, signal generation delay, Store event 1 pulse count in 0x11 & 0x10 and event 2 pulse count in 0x21 & 0x20, No of pulses/pulse

difference in 0x30, 0x31 and send the same to PortB and PortD. Using CCP1 and Timer 1. The logic is demonstrated in Fig P13.1

	ORG	0x0000	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start of Main program.
	ORG	0x0030	; Set Origin 0x0030 for Main program
START	CLRF	TRISB	; Make Port B as output
	CLRF	TRISD	; Make Port D as output
	BCF	TRISE,0	; Make RE0 pin as output to activate external ; signal generation device by clearing TRISE bit 0
	MOVLW	0x05	; WREG = 0x05
	MOVWF	CCP1CON	; CCP1CON = WREG = 0x05, Setting CCP1 for ; Capture mode with rising edge with 0 prescale
	BSF	TRISC,2	; Make CCP1/RC2 pin as input for capture
	MOVLW	0x00	; WREG = 0x00
	MOVWF	T3CON	; T3CON = 0x00, Cleared to Select Timer 1 as time ; source for CCP
	MOVWF	T1CON	; T1CON = 0x00, Select 8-bit read/write operations ; of Timer1, with Prescale 1, select internal main ; clock
UP	CLRF	TMR1H	; Initialize the Timer 1 high count reg TMR1H to ; 0x00
	CLRF	TMR1L	; Initialize the Timer 1 low count reg TMR1L to ; 0x00
	BCF	PIR1,CCP1IF	; Clear CCP1IF flag
	BSF	PORTE,0	; Set RE0 pin to activate external signal gen device
	BSF	T1CON,0 ;TMR1ON	; Start the Timer 1 to count the time pulses
L1	BTFSS	PIR1,CCP1IF	; Wait for first rising edge event. Check CCP1IF ; flag. Skip next instruction if event flag
	BRA	L1	; Branch to L1 if event flag not set
	MOVF	CCPR1H,W	; WREG = CCPR1H, Copy the latched Timer1 High
	MOVWF	0x11	; [0x11] = CCPR1H = WREG
	MOVF	CCPR1L,W	; WREG = CCPR1L, Copy the latched Timer1 Low
	MOVWF	0x10	; [0x10] = CCPR1L = WREG
	BCF	PIR1,CCP1IF	; Clear CCP1IF flag
L2	BTFSS	PIR1,CCP1IF	; Wait for second rising edge event. Check CCP1IF ; flag. Skip next instruction if event flag
	BRA	L2	; Branch to L2 if event flag not set
	BCF	T1CON,0 ;TMR1ON	; Stop the Timer 1.
	BCF	PORTE,0	; Clear RE0 pin to deactivate external signal ; generation device

MOVF	CCPR1H,W	; WREG = CCPR1H, Copy the latched Timer1 High
MOVWF	0x21	; [0x21] = CCPR1H = WREG
MOVF	CCPR1L,W	; WREG = CCPR1L, Copy the latched Timer1 Low
MOVWF	0x20	; [0x20] = CCPR1L = WREG
MOVF	0x10,0	; WREG = [0x10] i.e. Low byte - TMR1L of event 1 ; latched in to CCPR1L
SUBWF	0x20,0	; WREG = [0x20] - WREG (i.e. [0x10]), subtract low ; byte of first event count from low byte of second ; event count
MOVWF	0x30,0	; [0x30] = WREG i.e. Low byte Difference
MOVF	0x11,0	; WREG = [0x11] i.e. High byte - TMR1H of event 1 ; latched in to CCPR1H
SUBWFB	0x21,0	; WREG = [0x20] - WREG (i.e. [0x10]), subtract with ; borrow high byte of first event count from high ; byte of second event count
MOVWF	0x31,0	; [0x31] = WREG i.e. High byte Difference
MOVFF	0x30,PORTB	; Send Low pulse count to Port B, ; PORTB = CCPR1L = [0x30]
MOVFF	0x31,PORTD	; Send High pulse count to Port D, ; PORTD = CCPR1H = [0x31]
GOTO	UP	; Goto UP, Repeat this process continuously.
END		

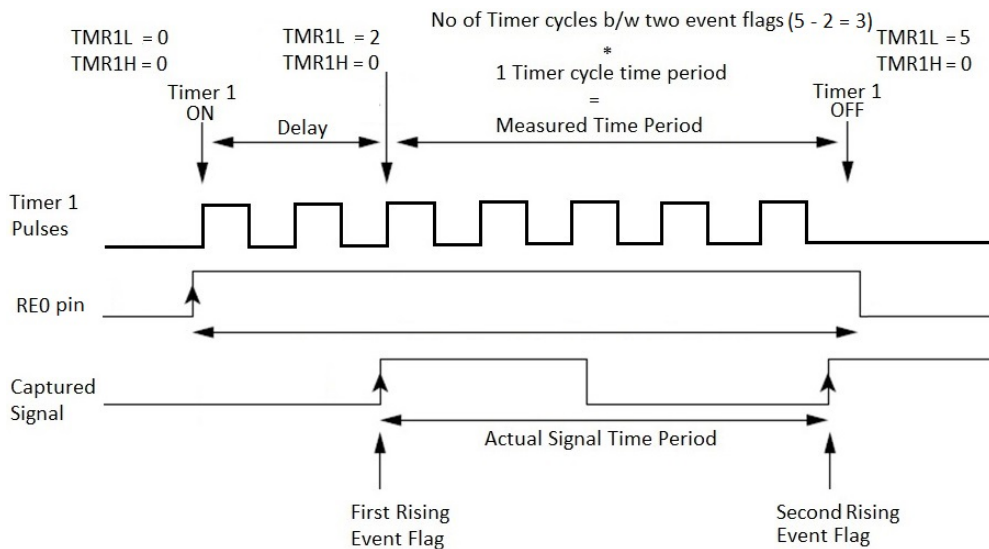


Fig P13.2: Logic demonstration of program 13.2.

Program 13.3: Program of 13.1 with interrupts

	ORG	0x0000	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start of Main program.
	ORG	0x0008	; Set Origin 0x0008 of High Priority Interrupt ; start address
	GOTO	CCP_ISR	; Go to CCP_ISR
	ORG	0x0030	; Set Origin 0x0030 for CCP_ISR
CCP_ISR	BTFSS	PIR1,CCP1IF	; Check if the interrupt is due to CCP1IF. Skip ; next instruction if its due to CCP1IF event flag.
	BRA	L1	; If not due to CCP1IF go to L1, do nothing, ; come out of ISR.
	BTFSC	0x00,0	; Check if its first/second CCP1IF event flag, by ; checking LSB bit of file register 0x00. Skip next ; instruction if clear, i.e. due to first event flag.
	BRA	L2	; Branch to L2 if its due to second event flag.
	BSF	0x00,0	; Set LSB bit of file register 0x00.
	CLRF	TMR1H	; Initialize Timer 1 high count register TMR1H ; to 0x00
	CLRF	TMR1L	; Initialize Timer 1 low count register TMR1L ; to 0x00
	BSF	T1CON,0	; Start the Timer 1 to see number of pulses till ; next event flag.
	BRA	L3	; Branch to L3 to come to end of ISR
L2	BCF	0x00,0	; Clear LSB bit of file register 0x00.
	CALL	DSEND	; Call Module DS, to get the Pulse count, store ; and send to ports.
L3	BCF	PIR1,CCP1IF	; Clear CCP1IF flag
L1	NOP		; No operation
	RETfie		; Return from ISR, Go back to main program
	ORG	0x0100	; Set Origin 0x0100 for module DSEND
DSEND	MOVF	CCPR1H,W	; WREG = CCPR1H, Copy the latched Timer1 ; High
	MOVWF	0x31	; [0x31] = CCPR1H = WREG
	MOVF	CCPR1L,W	; WREG = CCPR1L, Copy the latched Timer1 Low
	MOVWF	0x30	; [0x30] = CCPR1L = WREG
	MOVFF	0x30,PORTB	; Send Low pulse count to Port B, ; PORTB = CCPR1L = [0x30]
	MOVFF	0x31,PORTD	; Send High pulse count to Port D, ; PORTD = CCPR1H = [0x31]
	BCF	T1CON, 0	; Stop the Timer 1.
	RET		; Return from module DSEND

START	ORG	0x0500	; Set Origin 0x0500 for Main program
	CLRF	TRISB	; Make Port B as output
	CLRF	TRISD	; Make Port D as output
	MOVLW	0X05	; WREG = 0x05
	MOVWF	CCP1CON	; CCP1CON = WREG = 0x05, Setting CCP1 for ; Capture mode with rising edge with 0 prescale
	BSF	TRISC,2	; Make CCP1/RC2 pin as input for capture
	MOVLW	0x00	; WREG = 0x00
	MOVWF	T3CON	; T3CON = 0x00, Cleared to Select Timer 1 as ; time source for CCP
	MOVWF	T1CON	; T1CON =0x00, Select 8-bit read/write ; operations of Timer1, with Prescale 1, select ; internal main clock fosc/4
	BSF	INTON,GIE	; Enable Global interrupt
UP	BSF	INTON,PEIE	; Enable Peripheral interrupts
	BSF	RCON,IPEN	; Enable interrupt priorities
	BSF	IPR1,CCP1IP	; Make CCP1IF as high priority
	BSF	PIE1,CCP1IE	; Enable CCP1IF interrupt
	BCF	PIR1,CCP1IF	; Clear CCP1IF flag
	NOP		; No operation, Logic of optional alternate ; function may be placed here.
	GOTO	UP	; Goto UP, Repeat this process continuously.
	END		

Program 13.4: Program of 13.2 with interrupts

CCP_ISR	ORG	0x0000	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start of Main program.
	ORG	0X0008	; Set Origin 0x0008 of High Priority Interrupt ; start address
	GOTO	CCP_ISR	; Go to CCP_ISR
	ORG	0X0030	; Set Origin 0x0030 for CCP_ISR
	BTFSS	PIR1,CCP1IF	; Check if the interrupt is due to CCP1IF. Skip ; next instruction if its due to CCPIF event flag.
	BRA	L1	; If not due to CCP1IF go to L1, do nothing, ; come out of ISR.
	BTFSC	0x00,0	; Check if its first/second CCP1IF event flag, by ; checking LSB bit of file register 0x00. Skip next ; instruction if clear, i.e. due to first event flag.
	BRA	L2	; Branch to L2 if its due to second event flag.
	BSF	0x00,0	; Set LSB bit of file register 0x00.

L2	CALL	STORE	; Call Module STORE, to store CCPR1 data ; latched from first event timer value.
	BRA	L3	; Branch to L3 to come to end of ISR
	BCF	0x00,0	; Clear LSB bit of file register 0x00.
	CALL	DSEND	; Call Module DS, to get the Pulse count, store ; and send to ports.
L3	BCF	PIR1,CCP1IF	; Clear CCP1IF flag
L1	NOP		; No operation
	RETFIE		; Return from ISR, Go back to main program
DSEND	ORG	0X0100	; Set Origin 0x0100 for module DSEND
	MOVF	CCPR1H,W	; WREG = CCPR1H, Copy the latched Timer1 ; High
	MOVWF	0x21	; [0x21] = CCPR1H = WREG
	MOVF	CCPR1L,W	; WREG = CCPR1L, Copy the latched Timer1 Low
	MOVWF	0x20	; [0x20] = CCPR1L = WREG
	CALL	PSUB	; Call Module PSUB, Get pulse difference ; between first and second event.
	MOVFF	0x30,PORTB	; Send Low pulse count to Port B, ; PORTB = CCPR1L = [0x30]
	MOVFF	0x31,PORTD	; Send High pulse count to Port D, ; PORTD = CCPR1H = [0x31]
	BCF	T1CON,0	; Stop the Timer 1.
	CALL	SEQST	; Call Module SEQST, to restart sequence.
	RET		; Return from module DSEND
	ORG	0X0200	; Set Origin 0x0200 for module STORE
	MOVF	CCPR1H,W	; WREG = CCPR1H, Copy the latched Timer1 ; High
	MOVWF	0x11	; [0x11] = CCPR1H = WREG
STORE	MOVF	CCPR1L,W	; WREG = CCPR1L, Copy the latched Timer1 Low
	MOVWF	0x10	; [0x10] = CCPR1L = WREG
	RET		; Return from module STORE
	ORG	0X0300	; Set Origin 0x0300 for module PSUB
	MOVF	0x10,0	; WREG = [0x10] i.e. Low byte - TMR1L of event1 ; latched in to CCPR1L
	SUBWF	0x20,0	; WREG = [0x20] - WREG (i.e. [0x10]), subtract ; low byte of first event count from low byte of ; second event count
	MOVWF	0x30,0	; [0x30] = WREG i.e. Low byte Difference
	MOVF	0x11,0	; WREG = [0x11] i.e. High byte - TMR1H of event ; 1 latched in to CCPR1H
	SUBWFB	0x21,0	; WREG = [0x20] - WREG (i.e. [0x10]), subtract ; with borrow high byte of first event count ; from high byte of second event count

	MOVWF	0x31,0	; [0x31] = WREG i.e. High byte Difference
	RET		; Return from module PSUB
	ORG	0x0400	; Set Origin 0x0400 for module SEQST
SEQST	CLRF	TMR1H	; Initialize the Timer 1 high count register ; TMR1H to 0x00
	CLRF	TMR1L	; Initialize Timer 1 low count register TMR1L ; to 0x00
	BCF	PIR1,CCP1IF	; Clear CCP1IF flag
	BSF	PORTE,0	; Set RE0 pin to activate external signal ; generation device
	BSF	T1CON,0	; Start the Timer 1 to count the time pulses
	RET		; Return from module SEQST
	ORG	0x0500	; Set Origin 0x0500 for Main program
START	CLRF	TRISB	; Make Port B as output
	CLRF	TRISD	; Make Port D as output
	BCF	TRISE,0	; Make RE0 pin as output to activate external ; signal generation device by clearing TRISE bit 0
	MOVLW	0x05	; WREG = 0x05
	MOVWF	CCP1CON	; CCP1CON = WREG = 0x05, Setting CCP1 for ; Capture mode with rising edge with 0 prescale
	BSF	TRISC,2	; Make CCP1/RC2 pin as input for capture
	MOVLW	0x00	; WREG = 0x00
	MOVWF	T3CON	; T3CON = 0x00, Cleared to Select Timer 1 as ; time source for CCP
	BSF	INTON,GIE	; Enable Global interrupt
	BSF	INTON,PEIE	; Enable Peripheral interrupts
	BSF	RCON,IPEN	; Enable interrupt priorities
	BSF	IPR1,CCP1IP	; Make CCP1IF as high priority
	BSF	PIE1,CCP1IE	; Enable CCP1IF interrupt
	CALL	SEQST	; Call Module SEQST, to start sequence.
UP	NOP		; No operation, Logic of optional alternate ; function may be placed here.
	GOTO	UP	; Goto UP, Repeat this process continuously.
	END		

Program 13.5a): Determining the Pulse Width, Store No of pulses in 0x30,0x31 and send to PortB and PortD. Using CCP1 and Timer 1. The logic is similar to one demonstrated in Fig P13.1. If file register 0x01 contains 0, measure ON time; if 1, measure off time.

	ORG	0x0000	; Set Origin 0x0000 of Flash memory
	GOTO	ST	; Go to Start of Main program.
	ORG	0x0030	; Set Origin 0x0030 for Main program
ST	CLRF	TRISB	; Make Port B as output

	CLRF	TRISD	; Make Port D as output
	BTFSC	0x01,0	; Check 1st bit of file reg 0x01. Skip next ; instruction and select rising edge if cleared i.e. ; when [0x01]=0x00
	BRA	LRF1	; Go to LRF1, select falling edge if set i.e. when ; [0x01]=0x01
	MOVLW	0X05	; WREG = 0x05, with rising edge with 0 prescale
	BRA	LRF2	; Go to LRF2
LFR1	MOVLW	0X04	; WREG = 0x04, with falling edge with 0 prescale
LFR2	MOVWF	CCP1CON	; CCP1CON, Setting CCP1 for Capture mode ; rising/falling
	BSF	TRISC,2	; Make CCP1/RC2 pin as input for capture
	MOVLW	0x00	; WREG = 0x00
	MOVWF	T3CON	; T3CON = 0x00, Cleared to Select Timer 1 as ; time source for CCP
	MOVWF	T1CON	; T1CON =0x00, Select 8-bit read/write operations ; of Timer1, with Prescale 1, select int main clock
UP	CLRF	TMR1H	; Initialize Timer 1 high count reg TMR1H to 0x00
	CLRF	TMR1L	; Initialize Timer 1 low count register TMR1L to 0x00
	BCF	PIR1,CCP1IF	; Clear CCP1IF flag
L1	BTFSS	PIR1,CCP1IF	; Wait for first rising edge event. Check CCP1IF flag. ; Skip next instruction if event flag
	BRA	L1	; Branch to L1 if event flag not set
	BTG	CCP1CON,0	; Change to falling/opposite edge
	BCF	PIR1,CCP1IF	; Clear CCP1IF flag
	BSF	T1CON,0	; Start the Timer 1 to count the time pulses
L2	BTFSS	PIR1,CCP1IF	; Wait for first rising edge event. Check CCP1IF flag. ; Skip next instruction if event flag
	BRA	L2	; Branch to L1 if event flag not set
	BCF	T1CON,0	; Stop the Timer 1.
	MOVF	CCPR1H,W	; WREG = CCPR1H, Copy the latched Timer1 High
	MOVWF	0x31	; [0x31] = CCPR1H = WREG
	MOVF	CCPR1L,W	; WREG = CCPR1L, Copy the latched Timer1 Low
	MOVWF	0x30	; [0x30] = CCPR1L = WREG
	MOVFF	0x30,PORTB	; Send Low pulse count to Port B, ; PORTB = CCPR1L = [0x30]
	MOVFF	0x31,PORTD	; Send High pulse count to Port D, ; PORTD = CCPR1H = [0x31]
	BTG	CCP1CON,0	; Change to rising/opposite edge
	GOTO	UP	; Goto UP, Repeat this process continuously.
	END		

Program 13.5b): Program of 13.5a with interrupts

	ORG	0x0000	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start of Main program.
	ORG	0x0008	; Set Origin 0x0008 of High Priority Interrupt
			; start address
	GOTO	CCP_ISR	; Go to CCP_ISR
	ORG	0x0030	; Set Origin 0x0030 for CCP_ISR
CCP_ISR	BTFSS	PIR1,CCP1IF	; Check if the interrupt is due to CCP1IF. Skip
			; next instruction if its due to CCPIF event flag.
	BRA	L1	; If not due to CCP1IF go to L1, do nothing, come
			; out of ISR.
	BTFSC	0x00,0	; Check if its first/second CCP1IF event flag, by
			; checking LSB bit of file register 0x00. Skip next
			; instruction if it is clear, i.e. due to 1 st event flag.
	BRA	L2	; Branch to L2 if its due to second event flag.
	CLRF	TMR1H	; Initialize Timer 1 high count reg TMR1H to 0x00
	CLRF	TMR1L	; Initialize Timer 1 low count reg TMR1L to 0x00
	BSF	T1CON,0	; Start the Timer 1 to count the time pulses
	BSF	0x00,0	; Set LSB bit of file register 0x00.
	BRA	L3	; Branch to L3 to come to end of ISR
L2	BCF	0x00,0	; Clear LSB bit of file register 0x00.
	CALL	DSEND	; Call Module DS, to get the Pulse count, store
			; and send to ports.
L3	BTG	CCP1CON,0	; Change to opposite edge
	BCF	PIR1,CCP1IF	; Clear CCP1IF flag
L1	NOP		; No operation
	RETFIE		; Return from ISR, Go back to main program
	ORG	0x0100	; Set Origin 0x0100 for module DSEND
DSEND	MOVF	CCPR1H,W	; WREG = CCPR1H, Copy the latched Timer1 High
	MOVWF	0x31	; [0x31] = CCPR1H = WREG
	MOVF	CCPR1L,W	; WREG = CCPR1L, Copy the latched Timer1 Low
	MOVWF	0x30	; [0x30] = CCPR1L = WREG
	MOVFF	0x30,PORTB	; Send Low pulse count to Port B,
			; PORTB = CCPR1L = [0x30]
	MOVFF	0x31,PORTD	; Send High pulse count to Port D,
			; PORTD = CCPR1H = [0x31]
	BCF	T1CON,0	; Stop the Timer 1.
	RET		; Return from module DSEND
	ORG	0x0500	; Set Origin 0x0500 for Main program
START	CLRF	TRISB	; Make Port B as output
	CLRF	TRISD	; Make Port D as output
			; Check 1st bit of file reg 0x01. Skip next
			; instruction and select rising edge if cleared
	BTFSC	0x01,0	; i.e. when [0x01]=0x00
	BRA	LRF1	; Go to LRF1, select falling edge if set i.e.

			; when [0x01]=0x01
	MOVLW	0X05	; WREG = 0x05, with rising edge with 0 prescale
	BRA	LFR2	; Go to LFR2
LFR1	MOVLW	0X04	; WREG = 0x04, with falling edge with 0 prescale
			; CCP1CON, Setting CCP1 for Capture mode
LFR2	MOVWF	CCP1CON	; rising/falling
	BSF	TRISC,2	; Make CCP1/RC2 pin as input for capture
	MOVLW	0x00	; WREG = 0x00
	MOVWF	T3CON	; T3CON = 0x00, Cleared to Select Timer 1 as
			; time source for CCP
	MOVWF	T1CON	; T1CON =0x00, Select 8-bit read/write
			; operations of Timer1, with Prescale 1, select
			; internal main clock fosc/4
	BSF	INTON,GIE	; Enable Global interrupt
	BSF	INTON,PEIE	; Enable Peripheral interrupts
	BSF	RCON,IPEN	; Enable interrupt priorities
	BSF	IPR1,CCP1IP	; Make CCP1IF as high priority
	BSF	PIE1,CCP1IE	; Enable CCP1IF interrupt
	BCF	PIR1,CCP1IF	; Clear CCP1IF flag
UP	NOP		; No operation, Logic of optional alternate
			; function may be placed here.
	GOTO	UP	; Goto UP, Repeat this process continuously.
	END		

13.2.2 Compare Mode

The CCP module operating in compare mode makes use of the CCPxCON register to control the CCP module and 16-Bit CCPRxL (Low byte) & CCPRxH (High byte) register to hold the comparison value. It is used to compare with the 16-bit value of TMR1 (TMR1H & TMR1L) or TMR3 (TMR3H & TMR3L) register.

For the working of compare mode, timer1 and/or timer3 need to be running in timer mode or synchronized counter mode. The T3CON<6,3> bits T3CCP2 :T3CCP1 bits of timer3 control register are used to select which timer is used for the respective CCP module.

The CCPxIF flag bit of the respective CCP module is set when the match occurs. An interrupt could be generated when the CCPxIF flag is set if the corresponding interrupt enable bits are set.

When the timer TMR register and CCPxR register match occurs, the CCPx pin of the respective module can be driven to one of the following states based on the setting of CCPxM3:CCPxM0 bits of CCPxCON register.

- Driven high
- Driven low
- Toggle output (high-to-low or low-to-high)
- Remain unchanged

The CCPx pin needs to be configured as output by clearing the appropriate TRIS bit. When the software interrupt is chosen, the CCPx pin is not affected and only CCPx interrupt is generated if interrupt is enabled. The CCPx compare output latch is at a low level by default and is cleared when CCPxCON register is cleared. The compare output latch is different from port data latch. The compare mode could be used to generate a square waveform on CCPx Pin.

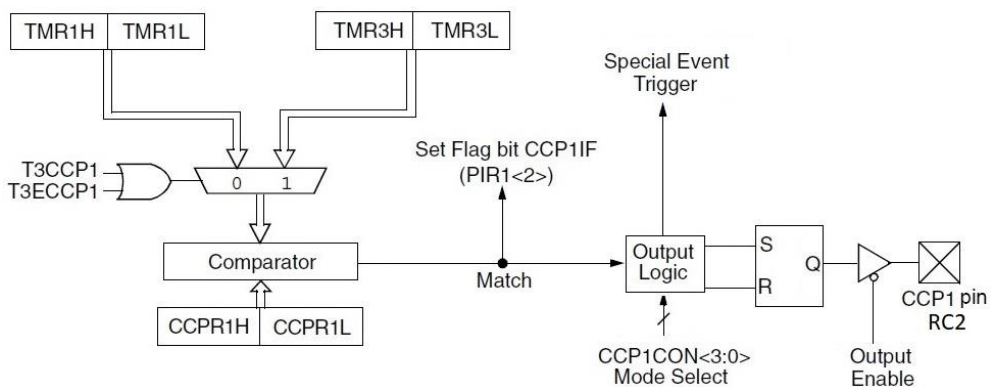


Fig 13.3 a) Block Diagram of Compare Mode in CCP1 of PIC18F458

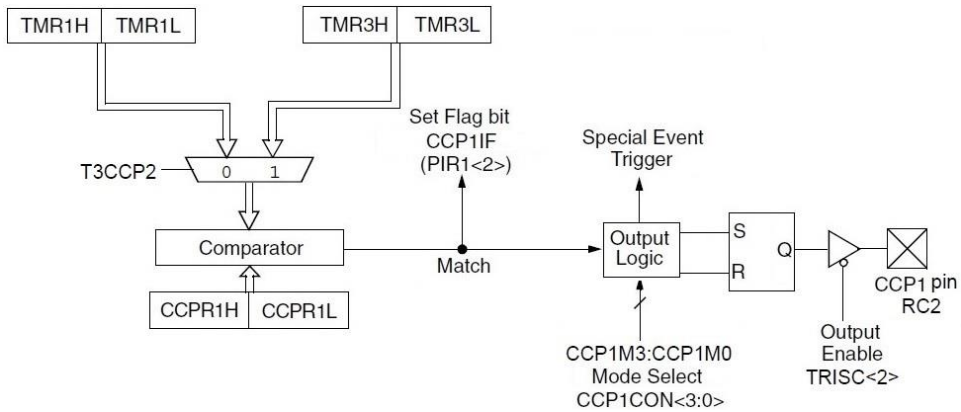


Fig 13.3 b) Block Diagram of Compare Mode: CCP1 – PIC18C601/801

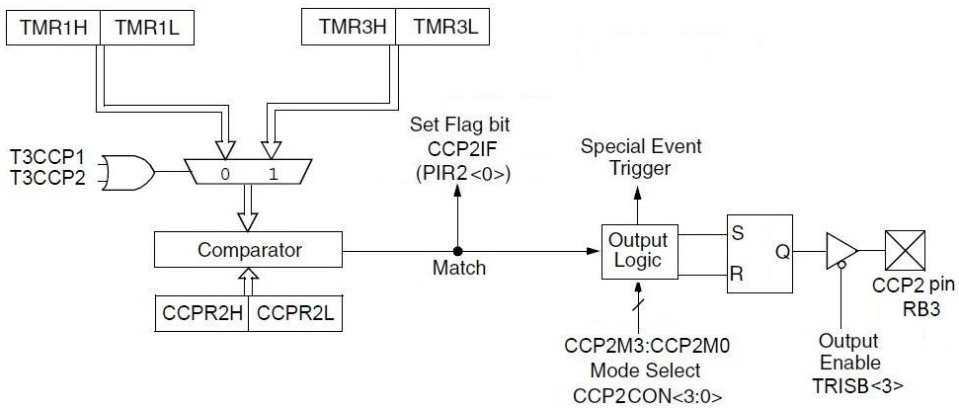
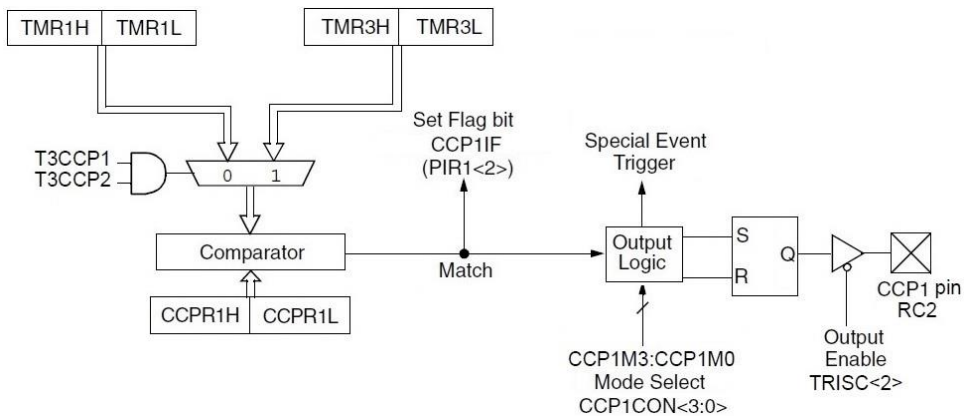


Fig 13.3 c) Block Diagram of Compare Mode: CCP2 – PIC18C601/801



Note: CCP2 pin is configurable on RC1/RE7/RB3
CCP3, CCP4 and CCP5 are available on RG0, RG3 and RG4 respectively.

Fig 13.3 d) Block Diagram of Compare Mode: CCP1 – PIC18F8720

The compare modes of CCP2, CCP3, CCP4 & CCP5 of PIC18f8720 uses equivalent structures based on the CCP & Timer interconnect configurations based on T3CCP<2:1> bits of T3CON register discussed earlier.

Configuring Setup steps for Compare Operation

1. Initialize the CCPxCON register to operate in Compare mode i.e CCPxCON<3:0> bits i.e. CCPxM3:CCPxM0.
2. Configure the respective CCPx pin as output pin by setting the respective TRIS bit.
3. Initialize Timer 3 control register T3CON<6,3> bits T3CCP2 :T3CCP1 bits to select Timer 1 or Timer 3 as source for the CCPx module.
4. Based on the selected timer Initialize the control register of Timer 1 (or Timer 3) with its prescale.
5. Initialize the CCPRxH & CCPRxL registers
6. Initialize Timer 1 register - (TMR1H & TMR1L) or Timer 3 (TMR3H & TMR3L) registers to appropriate value – typically zero.
7. Clear the Timer1 or Timer 3 and CCPxIF flags
8. Start/Enable the Timer1 (or Timer3)
9. Monitor CCPxIF or if required, Enable the CCPxIF interrupt and the required global interrupt.

Program 13.6: Generating square wave of 60% duty cycle with 10 msec off time for a 10Mhz clock using CCP in compare mode.

Sol: 10msec off time $\rightarrow T_{off} = T * 0.4 = 10\text{msec} \rightarrow \text{Period } T = 10\text{msec}/0.4 = 25\text{msec}$.

So ON time 60% of $T = T_{ON} = 0.6 * 25\text{msec} = 15\text{msec}$.

Clock Period = $T = 1/(f_{osc}/4) = 1/\{(10 \times 10^6)/4\} = 0.4 \mu\text{sec}$.

For ON time of 25 msec $\rightarrow \text{Count value} = 25\text{msec}/0.4 \mu\text{sec} = 62500 = 0xF424$

For ON time of 10 msec $\rightarrow \text{Count value} = 10\text{msec}/0.4 \mu\text{sec} = 25000 = 0x61A8$

	ORG	0x0000	; Set Origin 0x0000 of Flash memory
	GOTO	ST	; Go to Start of Main program.
	ORG	0x0030	; Set Origin 0x0030 for Main program
ST	MOVLW	0x02	; WREG = 0x02
	MOVWF	CCP1CON	; CCP1CON = WREG = 0x02, Set CCP1 for Compare, ; Toggle on match, use 0x08 to initialize pin low and ; set upon match, 0x09 for opposite.
	BCF	TRISC,2	; Make CCP1/RC2 pin output to get CCP Square WF
	BSF	TRISC,0	; Make T1CLKI/RC0 pin input to get input
	MOVLW	0x00	; WREG = 0x00

	MOVWF	T3CON	; T3CON = 0x00, Cleared to Select Timer 1 as time ; source for CCP
	MOVLW	0x02	; WREG = 0x02
	MOVWF	T1CON	; T1CON = 0x02, select 8-bit read/write operations ; of Timer1, with Prescale 1, select T1CLKI
UP	CLRF	TMR1H	; Initialize Timer 1 high count reg TMR1H to 0x00
	CLRF	TMR1L	; Initialize Timer 1 low count reg TMR1L to 0x00
	BTFSC	0x00,0	; Check if its first/second CCP1IF event flag, by ; checking LSB bit of file register 0x00. Skip next ; instruction if its clear, i.e. due to 1st event flag.
	BRA	L1	; Branch to L1, load CCPR1 reg with ON time value
	MOV	CCPR1H,0x61	; CCPR1H = 0x61, Load High byte of OFF time
	MOV	CCPR1L,0xA8	; CCPR1L = 0xA8, Load Low byte of OFF time
	BRA	L2	; Branch to L2 to continue OFF time part
L1	MOV	CCPR1H,0xF4	; CCPR1H = 0xF4, Load High byte of ON time
	MOV	CCPR1L,0x24	; CCPR1L = 0x24, Load Low byte of ON time
L2	BTG	0x00,0	; Toggle LSB bit of file register 0x00.
	BCF	PIR1,CCPIF	; Clear CCP1IF flag
	BSF	T1CON,0	; Start the Timer 1 to count the time pulses
L1	BTFSS	PIR1,CCP1IF	; Wait for first rising edge event. Check CCP1IF flag. ; Skip next instruction if event flag
	BRA	L1	; Branch to L1 if event flag not set
	BCF	T1CON,0	; Stop the Timer 1
	GOTO	UP	; Goto UP, Repeat this process continuously.
	END		

Program 13.7: Program of 13.6 with interrupts

	ORG	0x0000	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start of Main program.
	ORG	0x0008	; Set Origin 0x0008 of High Priority Interrupt ; start address
	GOTO	CCP_ISR	; Go to CCP_ISR
	ORG	0x0030	; Set Origin 0x0030 for CCP_ISR
CCP_ISR	BTFSS	PIR1,CCP1IF	; Check if the interrupt is due to CCP1IF. Skip ; next instruction if its due to CCPIF event flag.
	BRA	L1	; If not due to CCP1IF go to L1, do nothing, ; come out of ISR.
	BCF	T1CON,0	; Stop the Timer 1
	BTFSC	0x00,0	; Check if its first/second CCP1IF event flag, by ; checking LSB bit of file register 0x00. Skip ; next instruction if clear, ; i.e. due to first event flag.
	BRA	L2	; Branch to L1, to load CCPR1 registers with ON ; time value

L2	MOV	CCPR1H,0x61	; CCPR1H = 0x61, Load High byte of OFF time
	MOV	CCPR1L,0xA8	; CCPR1L = 0xA8, Load Low byte of OFF time
	BRA	L3	; Branch to L2 to continue OFF time part
	MOV	CCPR1H,0xF4	; CCPR1H = 0xF4, Load High byte of ON time
	MOV	CCPR1L,0x24	; CCPR1L = 0x24, Load Low byte of ON time
L3	BTG	0x00,0	; Toggle LSB bit of file register 0x00.
	CLRF	TMR1H	; Initialize Timer1 high count reg ; TMR1H to 0x00
L1	CLRF	TMR1L	; Initialize Timer1 low count reg TMR1L to 0x00
	BCF	PIR1,CCPIF	; Clear CCP1IF flag
	BSF	T1CON,0	; Start the Timer 1 to count the time pulses
	NOP		; No operation
	RETFIE		; Return from ISR, Go back to main program
START	ORG	0x0500	; Set Origin 0x0500 for Main program
	MOVLW	0x02	; WREG = 0x02
	MOVWF	CCP1CON	; CCP1CON = WREG = 0x02, Setting CCP1 for ; Compare, toggle on match, use 0x08 to ; initialize pin low and set upon match
	BCF	TRISC,2	; Make CCP1/RC2 pin as output to get CCP ; Square waveform
	BSF	TRISC,0	; Make T1CLKI/RC0 pin input to get input
	MOVLW	0x00	; WREG = 0x00
	MOVWF	T3CON	; T3CON = 0x00, Cleared to Select Timer 1 as ; time source for CCP
	MOVLW	0x02	; WREG = 0x02
	MOVWF	T1CON	; T1CON = 0x02, Select 8-bit read/write ; operations of Timer1, with Prescale 1, select ; T1CLKI
UP	BSF	INTON,GIE	; Enable Global interrupt
	BSF	INTON,PEIE	; Enable Peripheral interrupts
	BSF	RCON,IPEN	; Enable interrupt priorities
	BSF	IPR1,CCP1IP	; Make CCP1IF as high priority
	BSF	PIE1,CCP1IE	; Enable CCP1IF interrupt
	BCF	PIR1,CCP1IF	; Clear CCP1IF flag
	BSF	T1CON,0	; Start the Timer 1 to count the time pulses
	NOP		; No operation, Logic of optional alternate ; function may be placed here.
	GOTO	UP	; Goto UP, Repeat this process continuously.
	END		

Trigger special event

When a CCP module is configured for use in compare mode with trigger special event, an internal hardware trigger is generated after a compare match and the following occurs in the respective device module.

- In PIC18F458:

- In CCP1, CCP1IF bit is set on compare; resets TMR1 or TMR3; Timer 1 or Timer 3 interrupt flags are set; starts A/D conversion if A/D module is enabled.
- In ECCP1, ECCP1IF bit is set on compare; resets TMR1 or TMR3; Timer 1 or Timer 3 interrupt flags are not set; starts A/D conversion if A/D module is enabled.
- In PIC18C601/801:
 - In CCP1, CCP1IF bit is set on compare; resets TMR1 which makes CCPR1 to be programmable 16-bit period register for timer 1; Timer 1 or Timer 3 interrupt flags are set.
 - In CCP2, CCP2IF bit is set on compare; resets TMR1 or TMR3; Timer 1 or Timer 3 interrupt flags are not set; starts A/D conversion if A/D module is enabled.
- In PIC18F8720:
 - CCPxIF bit is set for the respective CCP module.
 - For CCP1
 - Timer 1 or Timer 3 interrupt flags are set;
 - Resets Timer 1 or Timer 3 on compare event.
 - Makes CCPR1 to be programmable 16-bit period register for Timer 1 or Timer 3.
 - For CCP2
 - Timer 1 or Timer 3 interrupt flags are not set;
 - Resets Timer 1 or Timer 3 on compare event.
 - start an A/D conversion if the A/D module is enabled.
 - For other modules
 - CCPx pin is unaffected and is configured as I/O port. (Same as CCPxM<3:0> = 1010, above)

Table 13.4 Interaction of CCP1 & ECCP1 in PIC18F458, CCPx & CCPy in PIC18C801/F8720.

CCPx Mode	CCPy/ ECCPz Mode	Interaction
Capture	Capture	TMR1 or TMR3 time base. Time base can be different for each CCP.
Capture	Compare	The compare could be configured for the special event trigger which clears either TMR1 or TMR3, depending upon which time base is used.
Compare	Compare	The compare(s) could be configured for the special event trigger which clears TMR1 or TMR3, depending upon which time base is used.
PWM	PWM	The PWMs will have the same frequency and update rate (TMR2 interrupt).
PWM	Capture	None.
PWM	Compare	None.

13.2.3 PWM Mode

PWM – Pulse Width Modulation is a technique for delivering a controlled average power by an electrical signal. The average value of voltage/current fed to a load is achieved by switching the supply ON or OFF and setting the proportion of ON time or OFF time. The time period is defined as sum of ON time and OFF time in one cycle. The Duty Cycle is defined as the ratio of ON time to the total time or time period. PWM waveforms for different duty cycles is presented in fig 13.4a and sample waveform with indication of Duty cycle, period and register is shown in fig 13.4b. PWM signals are generally used in speed control of DC motors.

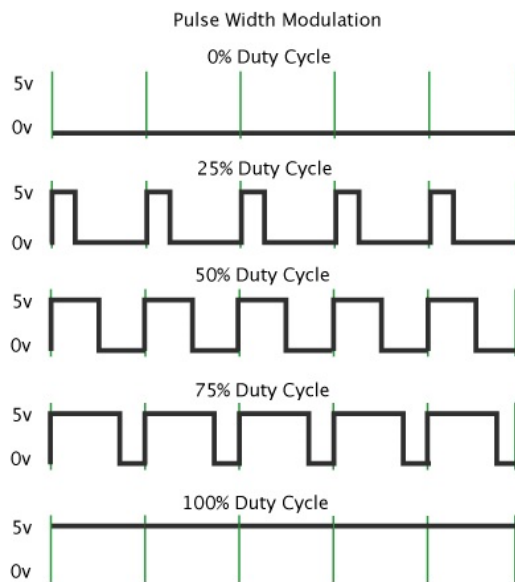


Fig 13.4 a) Waveforms for different Duty cycles

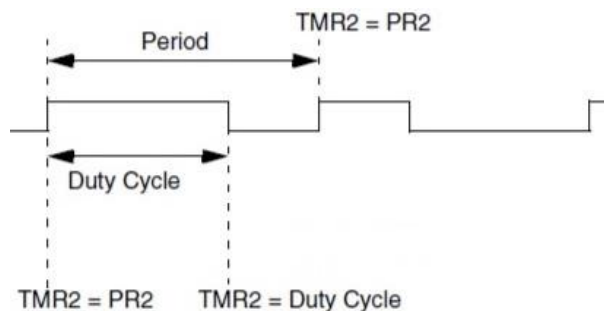


Fig 13.4 b) Sample waveform with indication of Duty Cycle, Period and registers.

Fig 13.4 PWM Output Waveform

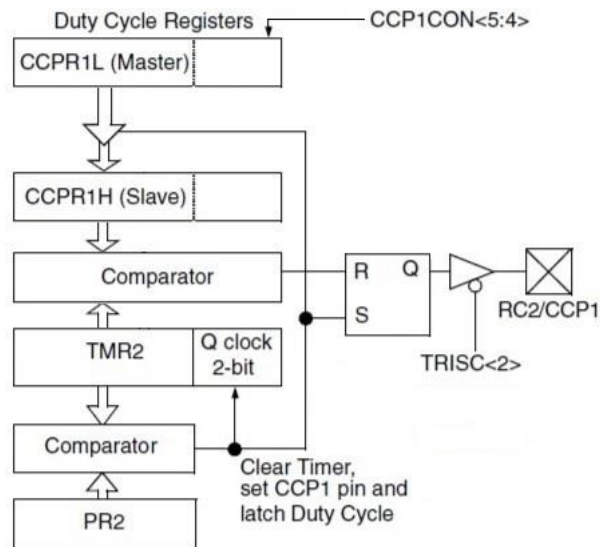
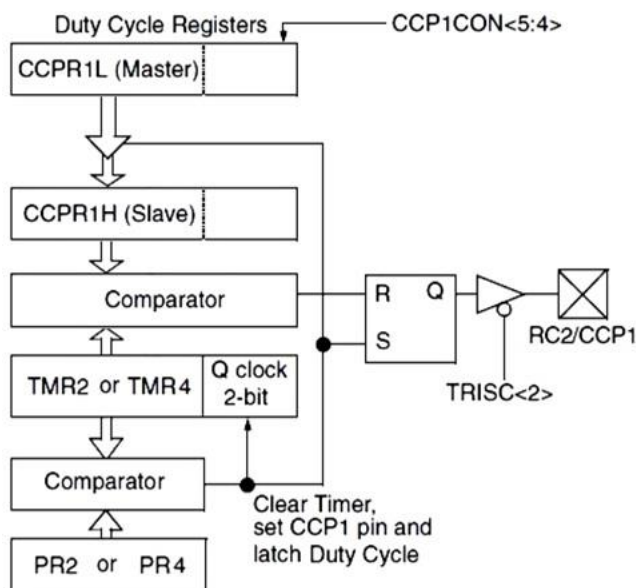


Fig.13.5 a) Block Diagram of CCP's PWM Mode in PIC18F458, PIC18C601/801.



Note: CCP2 pin is configurable on RC1/RE7/RB3

CCP3, CCP4 and CCP5 are available on RG0, RG3 and RG4 respectively.

Fig.13.5 b) Block Diagram of CCP's PWM Mode in PIC18f8720.

T3CCP<2:1> bits select TM2 or TM4.

In PWM mode, the CCP module makes of CCPRxL to load the duty cycle value and the decimal point value is set by the CCPxCON<5:4> bits - DC1B1 DC1B0. The PR2 register is used to set the time period value. Whenever CCPRxL is greater than PR2, 100% duty cycle is achieved and the CCP1 pin always stays high.

The block diagram of PIC18s CCP1 module in PWM mode is shown in fig 13.5a for PIC18F458, PIC18C601/801 and fig 13.5b for PIC18F8720. An equivalent structure is used in CCP2 module of PIC18C601/801. The PWM modes in CCP2, CCP3, CCP4 & CCP5 modules of PIC18F8720 also uses equivalent structures based on the CCP & Timer interconnect configurations T3CCP<2:1> bits of T3CON register discussed earlier.

CCPRxL is written with the duty cycle value and PR2 (or PR4) is written with a PWM Period value. When the TMR2 (or TMR4) matches the PR2 (or PR4) value, the following occur on the next increment cycle.

- the TMR2 (or TMR4) is cleared,
- sets the CCPx pin (except when duty cycle is 0%) and
- the duty cycle value of CCPRxL is latched into CCPRxH.

When the timer TMR2 (or TMR4) reaches the CCPRxH the CCPx pin is reset. The respective waveform is shown in fig 13.4b. For the working of PWM mode the timer2 or timer 4 need to be running in timer mode or synchronized counter mode. Whenever time period is complete (match of TMR and PR occurs) the TMR register is cleared. Hence in a PIC18F458 there is only one period value for both CCP and ECCP modules. Similarly, PIC18C601/801 also has only one period value for both CCP1 and CCP2 modules. Also the PIC18F8720 can have at the most 2 period values for the 5 CCP modules based on the selected timers.

Table 13.5: PWM decimal points

DC1B1	DC1B0	Decimal points
0	0	0
0	1	0.25
1	0	0.5
1	1	0.75

In PWM mode the CCPx module generates a PWM output with a 10-bit resolution on the respective CCPx pin. To generate a 10-bit resolution one of the 10-bit value is formed by concatenating the TMR2 register with 2-bit internal Q clock. The other 10-bit value is formed by concatenating the CCPRxH value (high 8-bit) with the two decimal point LSB's CCPxCON<5:4> bits - DC1B1 DC1B0. The CCPRxL, DC1B1 and DC1B0 can be written any time but the 10-bit value is loaded to CCPR1H only when the PR2 and TMR2 (or TMR4) match occurs (i.e. Period complete). When the timer TMR2 (or TMR4) with internal 2-bit Q clock reaches the CCPRxH with 2-bit value the CCPx pin is reset. To use the PWM output the respective port pin need to be configured as output by clearing the TRIS bit. For CCP1 it is TRISC<2> bit. The appropriate bits for different CCP modules is shown earlier in Table 13.3. The PWM frequency is defined as the inverse of PWM period. i.e.

$$f_{PWM} = 1/T_{PWM}. \quad (\text{Eq 13.1})$$

The PWM period is set by writing the respective value in PR2 (or PR4) register. It is given by the following formula:

$$\text{PWM Period (T}_{\text{PWM}}) = [\text{PR2} + 1] \times 4 \times \text{TOSC} \times (\text{TMR2 Prescale Value}) \quad (\text{Eq 13.2})$$

N = TMR2 Prescale Value

$$\text{PWM Period (T}_{\text{PWM}}) = [\text{PR2} + 1] \times 4 \times \text{TOSC} \times \text{N} \quad (\text{Eq 13.2a})$$

$$\text{PR2} = [\text{T}_{\text{PWM}} / \{4 \times \text{T}_{\text{OSC}} \times \text{N}\}] - 1 \quad (\text{Eq 13.2b})$$

$$\text{PR2} = [\text{f}_{\text{osc}} / \{4 \times \text{f}_{\text{PWM}} \times \text{N}\}] - 1 \quad (\text{Eq 13.2c})$$

Here N can take values of 1, 4 or 16 and PR2 can range from 0 to 255. The PWM Duty cycle is set by writing the respective value in CCPRxL register and writing to CCPxCON<5:4> bits. It is given by the following formula:

$$\text{PWM Duty Cycle} = (\text{CCPR1L:CCP1CON<5:4>}) \times \text{TOSC} \times (\text{TMR2 Prescale Value}) \quad (\text{Eq 13.3})$$

The CCPRxL acts as a duty cycle register since it is always a percentage of the PR2 (or PR4) register. As an example, if PR2 is having a period value of 30, and a duty cycle of 20% is required, the CCPRxL is to be loaded with a value of 6 since 6 is 20% of 30. If 25% duty cycle is required, the CCPRxL is to be loaded with a value of 7.5 but CCPRxL is only loaded with 7 and the decimal point 0.5 is loaded through the value of 10 in the CCPxCON<5:4> bits DC1B1:DC1B0.

The relation between the maximum PWM resolution for a given PWM frequency is given by the following expression.

$$\text{PWM Resolution (max)} = \{ [\log(\text{F}_{\text{osc}} / \text{F}_{\text{PWM}})] / \log(2) \} \text{ bits.}$$

Table 13.6: Example PWM frequencies and resolution

PWM Frequency	2.44 kHz	9.76 kHz	39.06 kHz	156.3 kHz	312.5 kHz	416.6 kHz
Timer Prescaler (1, 4, 16)	16	4	1	1	1	1
PR2 Value	0FFh	0FFh	0FFh	3Fh	1Fh	17h
Maximum Resolution (bits)	10	10	10	8	7	5.5

Source: Microchip Document: PIC18FXX8 Data Sheet - DS41159E

Example 13.1: Find the minimum and maximum PWM time period and frequency for an oscillator frequency of 10 MHz.

Sol:

When N = 16

$$\text{T}_{\text{PWM1 max}} = [255+1] \times 4 \times 0.1 \times 10^{-6} \times 16 = 256 \times 0.4 \times 10^{-6} \times 16 = 1638.4 \times 10^{-6} \text{ Sec.}$$

$$T_{PWM2} \min = [0+1] \times 4 \times 0.1 \times 10^{-6} \times 16 = 1 \times 0.4 \times 10^{-6} \times 16 = 6.4 \times 10^{-6} \text{ Sec.}$$

$$f_{PWM1} \min = 1/T_{PWM1} \max = 610 \text{ Hz}$$

$$f_{PWM2} \max = 1/T_{PWM1} \min = 156.25 \text{ KHz.}$$

When N = 4

$$T_{PWM1} \max = [255+1] \times 4 \times 0.1 \times 10^{-6} \times 4 = 256 \times 0.4 \times 10^{-6} \times 4 = 409.6 \times 10^{-6} \text{ Sec.}$$

$$T_{PWM2} \min = [0+1] \times 4 \times 0.1 \times 10^{-6} \times 4 = 1 \times 0.4 \times 10^{-6} \times 4 = 1.6 \times 10^{-6} \text{ Sec.}$$

$$f_{PWM1} \min = 1/T_{PWM1} \max = 2.44 \text{ KHz.}$$

$$f_{PWM2} \max = 1/T_{PWM1} \min = 625 \text{ KHz.}$$

When N = 1

$$T_{PWM1} \max = [255+1] \times 4 \times 0.1 \times 10^{-6} \times 1 = 256 \times 0.4 \times 10^{-6} \times 1 = 102.4 \times 10^{-6} \text{ Sec.}$$

$$T_{PWM2} \min = [0+1] \times 4 \times 0.1 \times 10^{-6} \times 1 = 1 \times 0.4 \times 10^{-6} \times 1 = 0.4 \times 10^{-6} \text{ Sec.}$$

$$f_{PWM1} \min = 1/T_{PWM1} \max = 9.765 \text{ KHz.}$$

$$f_{PWM2} \max = 1/T_{PWM1} \min = 2.5 \text{ MHz.}$$

Example 13.2 Find the minimum and maximum PWM time period and frequency for an oscillator frequency of 20 MHz.

Sol:

When N = 16

$$T_{PWM1} \max = [255+1] \times 4 \times 0.05 \times 10^{-6} \times 16 = 256 \times 0.2 \times 10^{-6} \times 16 = 819.2 \times 10^{-6} \text{ Sec.}$$

$$T_{PWM2} \min = [0+1] \times 4 \times 0.05 \times 10^{-6} \times 16 = 1 \times 0.2 \times 10^{-6} \times 16 = 3.2 \times 10^{-6} \text{ Sec.}$$

$$f_{PWM1} \min = 1/T_{PWM1} \max = 1.22 \text{ KHz.}$$

$$f_{PWM2} \max = 1/T_{PWM1} \min = 312 \text{ KHz.}$$

When N = 4

$$T_{PWM1} \max = [255+1] \times 4 \times 0.05 \times 10^{-6} \times 4 = 256 \times 0.2 \times 10^{-6} \times 4 = 204.8 \times 10^{-6} \text{ Sec.}$$

$$T_{PWM2} \min = [0+1] \times 4 \times 0.05 \times 10^{-6} \times 4 = 1 \times 0.2 \times 10^{-6} \times 4 = 0.8 \times 10^{-6} \text{ Sec.}$$

$$f_{PWM1} \min = 1/T_{PWM1} \max = 4.88 \text{ KHz.}$$

$$f_{PWM2} \max = 1/T_{PWM1} \min = 1.25 \text{ MHz.}$$

When N = 1

$$T_{PWM1} \max = [255+1] \times 4 \times 0.05 \times 10^{-6} \times 1 = 256 \times 0.2 \times 10^{-6} \times 1 = 51.2 \times 10^{-6} \text{ Sec.}$$

$$T_{PWM2} \min = [0+1] \times 4 \times 0.05 \times 10^{-6} \times 1 = 1 \times 0.2 \times 10^{-6} \times 1 = 0.2 \times 10^{-6} \text{ Sec.}$$

$$f_{PWM1} \min = 1/T_{PWM1} \max = 19.5 \text{ KHz.}$$

$$f_{PWM2} \max = 1/T_{PWM1} \min = 5 \text{ MHz.}$$

Table E13.1: Range of possible PWM frequencies

Oscillation frequency	TMR2 Prescale	Possible PWM frequency Range
For 10MHz fosc: Summary from Example 13.1	N=16	610Hz to 156.25KHz
	N=4	2.44KHz to 625KHz
	N=1	9.765KHz to 2.5MHz
	N = 1, 4, 16	610Hz to 2.5MHz
For 20MHz fosc: Summary from Example 13.2	N=16	1.22KHz to 312KHz
	N=4	4.88KHz to 1.25MHz
	N=1	19.5KHz to 5MHz
	N = 1, 4, 16	1.22KHz to 5MHz

Example 13.3: Find the value of PR2 which could generate the following PWM frequencies with an oscillator frequency of 10 MHz. Also cross check the generated frequencies.

- a) 500 Hz b) 1.0016 KHz c) 5 KHz

Sol:

From Eq 13.2a

$$(T_{PWM}) = [PR2 + 1] \times 4 \times TOSC \times N$$

From Eq 13.2c

$$PR2 = \left[\frac{f_{osc}}{4 \times f_{PWM} \times N} \right] - 1 \quad \rightarrow \quad PR2 = \left[\frac{10M}{4fN} \right] - 1$$

- a) 500Hz

$$\text{For } N=1 \quad \rightarrow \quad PR2 = \left[\frac{10MHz}{4 \times 500Hz \times 1} \right] - 1 = 5000 - 1 = 4999.$$

It is larger than 8-bit value 255 that can fit in PR2. So it is not possible.

$$\text{For } N=4 \quad \rightarrow \quad PR2 = \left[\frac{10MHz}{4 \times 500Hz \times 4} \right] - 1 = 1250 - 1 = 1249.$$

It is larger than 8-bit value 255 that can fit in PR2. So it is not possible.

$$\text{For } N=16 \quad \rightarrow \quad PR2 = \left[\frac{10MHz}{4 \times 500Hz \times 16} \right] - 1 = 312.5 - 1 = 311.5.$$

It is larger than 8-bit value 255 that can fit in PR2. So it is not possible.

So a PWM frequency of 500Hz is not possible with Timer 2 with osc frequency of 10MHz.

- b) 1.0016 KHz

$$\text{For } N=1 \quad \rightarrow \quad PR2 = \left[\frac{10MHz}{4 \times 1001.6Hz \times 1} \right] - 1 = 2504 - 1 = 2503.$$

It is larger than 8-bit value 255 that can fit in PR2. So it is not possible.

$$\text{For } N=4 \quad \rightarrow \quad PR2 = \left[\frac{10MHz}{4 \times 1001.6Hz \times 4} \right] - 1 = 624 - 1 = 623.$$

It is larger than 8-bit value 255 that can fit in PR2. So it is not possible.

$$\text{For } N=16 \quad \rightarrow \quad PR2 = \left[\frac{10MHz}{4 \times 1001.6Hz \times 16} \right] - 1 = 156 - 1 = 155$$

The value is smaller than 8-bit value 255 and can fit in PR2.

$$T_{PWM1} = [155+1] \times 4 \times 0.1 \times 10^{-6} \times 16 = 156 \times 6.4 \times 10^{-6} = 998.4 \mu\text{Sec}$$

$$f_{PWM1} = 1/T_{PWM1} = 1.0016 \text{ KHz.}$$

c) 5 KHz

$$\text{For } N=1 \rightarrow PR2 = [10\text{MHz}/\{4 \times 5\text{KHz} \times 1\}] - 1 = 500 - 1 = 499.$$

It is larger than 8-bit value 255 that can fit in PR2. So it is not possible.

$$\text{For } N=4 \rightarrow PR2 = [10\text{MHz}/\{4 \times 5\text{KHz} \times 4\}] - 1 = 125 - 1 = 124.$$

The value is smaller than 8-bit value 255 and can fit in PR2.

$$T_{PWM1} = [124+1] \times 4 \times 0.1 \times 10^{-6} \times 4 = 125 \times 1.6 \times 10^{-6} = 200 \mu\text{Sec}$$

$$f_{PWM1} = 1/T_{PWM1} = 5 \text{ KHz.}$$

$$\text{For } N=16 \rightarrow PR2 = [10\text{MHz}/\{4 \times 5\text{KHz} \times 16\}] - 1 = 31.25 - 1 = 30.25$$

The value is smaller than 8-bit value 255 and can fit in PR2.

$$T_{PWM1} = [30+1] \times 4 \times 0.1 \times 10^{-6} \times 16 = 31 \times 6.4 \times 10^{-6} = 198.4 \mu\text{Sec}$$

$$T_{PWM2} = [31+1] \times 4 \times 0.1 \times 10^{-6} \times 16 = 32 \times 6.4 \times 10^{-6} = 204.8 \mu\text{Sec}$$

$$f_{PWM1} = 1/T_{PWM1} = 5.04 \text{ KHz.}$$

$$f_{PWM2} = 1/T_{PWM2} = 4.883 \text{ KHz.}$$

Since $N = 4$ gives accurate 5KHz, $N = 4$ is considered to be the best prescale.

Example 13.4: Find the PWM frequencies which could be generated with an oscillator frequency of 20 MHz for the following PR values for different prescales.

a) 32 b) 64 c) 127 d) 191

Sol:

From Eq 13.2a

$$(T_{PWM}) = [PR2 + 1] \times 4 \times TOSC \times N$$

a) 32

$$\text{For } N=1 \quad T_{PWM} = [32+1] \times 4 \times 0.05 \times 10^{-6} \times 1 = 6.6 \mu\text{Sec}$$

$$f_{PWM} = 1/T_{PWM} = 151.515 \text{ KHz.}$$

$$\text{For } N=4 \quad T_{PWM} = [32+1] \times 4 \times 0.05 \times 10^{-6} \times 4 = 26.4 \mu\text{Sec}$$

$$f_{PWM} = 1/T_{PWM} = 37.88 \text{ KHz.}$$

$$\text{For } N=16 \quad T_{PWM} = [32+1] \times 4 \times 0.05 \times 10^{-6} \times 16 = 105.6 \mu\text{Sec}$$

$$f_{PWM} = 1/T_{PWM} = 9.47 \text{ KHz.}$$

b) 64

$$\text{For } N=1 \quad T_{PWM} = [64+1] \times 4 \times 0.05 \times 10^{-6} \times 1 = 13 \mu\text{Sec}$$

$$f_{PWM} = 1/T_{PWM} = 76.92 \text{ KHz.}$$

$$\text{For } N=4 \quad T_{PWM} = [64+1] \times 4 \times 0.05 \times 10^{-6} \times 4 = 52 \mu\text{Sec}$$

$$f_{PWM} = 1/T_{PWM} = 19.23 \text{ KHz.}$$

$$\text{For } N=16 \quad T_{PWM} = [64+1] \times 4 \times 0.05 \times 10^{-6} \times 16 = 208 \mu\text{Sec}$$

$$f_{PWM} = 1/T_{PWM} = 4.8 \text{ KHz.}$$

c) 127

$$\text{For } N = 1 \quad T_{PWM} = [127+1] \times 4 \times 0.05 \times 10^{-6} \times 1 = 25.6 \mu\text{Sec}$$

$$f_{PWM} = 1/T_{PWM} = 39.06 \text{ KHz.}$$

$$\text{For } N = 4 \quad T_{PWM} = [127+1] \times 4 \times 0.05 \times 10^{-6} \times 4 = 102.4 \mu\text{Sec}$$

$$f_{PWM} = 1/T_{PWM} = 9.76 \text{ KHz.}$$

$$\text{For } N = 16 \quad T_{PWM} = [127+1] \times 4 \times 0.05 \times 10^{-6} \times 16 = 409.6 \mu\text{Sec}$$

$$f_{PWM} = 1/T_{PWM} = 2.44 \text{ KHz.}$$

d) 191

$$\text{For } N = 1 \quad T_{PWM} = [191+1] \times 4 \times 0.05 \times 10^{-6} \times 1 = 38.4 \mu\text{Sec}$$

$$f_{PWM} = 1/T_{PWM} = 26.04 \text{ KHz.}$$

$$\text{For } N = 4 \quad T_{PWM} = [191+1] \times 4 \times 0.05 \times 10^{-6} \times 4 = 153.6 \mu\text{Sec}$$

$$f_{PWM} = 1/T_{PWM} = 6.51 \text{ KHz.}$$

$$\text{For } N = 16 \quad T_{PWM} = [191+1] \times 4 \times 0.05 \times 10^{-6} \times 16 = 614.4 \mu\text{Sec}$$

$$f_{PWM} = 1/T_{PWM} = 1.63 \text{ KHz.}$$

Example 13.5: Find the value of PR2, CCP1RL and DC1B1:DC1B0 which could generate the following PWM frequencies with an oscillator frequency of 10 MHz for duty cycles of 25%, 40%, 50%, 60% and 75%

a) 1.0016 KHz b) 5 KHz c) 20 KHz d) 25 KHz e) 80KHz

Sol:

From Eq 13.2a

$$(T_{PWM}) = [PR2 + 1] \times 4 \times TOSC \times N$$

From Eq 13.2c

$$PR2 = [f_{osc} / \{4 \times f_{PWM} \times N\}] - 1 \quad \rightarrow \quad PR2 = [10M / \{4fN\}] - 1$$

a) 1.0016 KHz

$$\text{For } N = 16 \rightarrow PR2 = [10\text{MHz} / \{4 \times 1.0016 \text{ KHz} \times 16\}] - 1 = 156 - 1 = 155$$

$$T_{PWM1} = [155+1] \times 4 \times 0.1 \times 10^{-6} \times 16 = 156 \times 6.4 \times 10^{-6} = 998.4 \mu\text{Sec}$$

$$f_{PWM1} = 1/T_{PWM1 \text{ max}} = 1.0016 \text{ KHz.}$$

$$PR2 = 155;$$

$$\text{For 25\% Duty cycle} \quad \text{Count} = 38.75 \quad \text{CCPR1L} = 38. \quad \text{DC1B1:DC1B0} = 11 (0.75)$$

$$\text{For 40\% Duty cycle} \quad \text{Count} = 62 \quad \text{CCPR1L} = 62. \quad \text{DC1B1:DC1B0} = 00 (0)$$

$$\text{For 50\% Duty cycle} \quad \text{Count} = 77.5 \quad \text{CCPR1L} = 77. \quad \text{DC1B1:DC1B0} = 10 (0.5)$$

$$\text{For 60\% Duty cycle} \quad \text{Count} = 93 \quad \text{CCPR1L} = 93. \quad \text{DC1B1:DC1B0} = 00 (0)$$

For 75% Duty cycle Count = 116.25 CCPR1L = 116. DC1B1:DC1B0 = 01 (0.25)

b) 4.9603 KHz

For **N= 4** $\rightarrow PR2 = [10\text{MHz}/\{4 \times 4.9603 \text{ KHz} \times 4\}] - 1 = 126 - 1 = 125.$

$T_{PWM1} = [125+1] \times 4 \times 0.1 \times 10^{-6} \times 4 = 126 \times 1.6 \times 10^{-6} = 201.6 \mu\text{Sec}$

$f_{PWM1} = 1/T_{PWM1} = 4.9603 \text{ KHz}.$

PR2 = 125;

For 25% Duty cycle Count = 31.25 CCPR1L = 31. DC1B1:DC1B0 = 01 (0.25)

For 40% Duty cycle Count = 50 CCPR1L = 50. DC1B1:DC1B0 = 00 (0)

For 50% Duty cycle Count = 62.5 CCPR1L = 62. DC1B1:DC1B0 = 10 (0.5)

For 60% Duty cycle Count = 75 CCPR1L = 75. DC1B1:DC1B0 = 00 (0)

For 75% Duty cycle Count = 93.75 CCPR1L = 93. DC1B1:DC1B0 = 11 (0.75)

c) 25 KHz

For **N= 1** $\rightarrow PR2 = [10\text{MHz}/\{4 \times 25 \text{ KHz} \times 1\}] - 1 = 100 - 1 = 99.$

$T_{PWM1} = [99+1] \times 4 \times 0.1 \times 10^{-6} \times 1 = 100 \times 0.4 \times 10^{-6} = 40 \mu\text{Sec}$

$f_{PWM1} = 1/T_{PWM1} = 25 \text{ KHz}.$

PR2 = 99;

For 25% Duty cycle Count = 24.75 CCPR1L = 24. DC1B1:DC1B0 = 11 (0.75)

For 40% Duty cycle Count = 39.6 CCPR1L = 39. DC1B1:DC1B0 = 10 (0.5)

For 50% Duty cycle Count = 49.5 CCPR1L = 49. DC1B1:DC1B0 = 10 (0.5)

For 60% Duty cycle Count = 59.4 CCPR1L = 59. DC1B1:DC1B0 = 10 (0.5)

For 75% Duty cycle Count = 74.25 CCPR1L = 74. DC1B1:DC1B0 = 01 (0.25)

40% and 60% Duty is not achievable as 0.6 and 0.4 cannot be set by 2-bit decimal value.

For **N= 4** $\rightarrow PR2 = [10\text{MHz}/\{4 \times 25 \text{ KHz} \times 4\}] - 1 = 25 - 1 = 24.$

$T_{PWM1} = [24+1] \times 4 \times 0.1 \times 10^{-6} \times 4 = 25 \times 1.6 \times 10^{-6} = 40 \mu\text{Sec}$

$f_{PWM1} = 1/T_{PWM1} = 25 \text{ KHz}.$

PR2 = 24;

For 25% Duty cycle Count = 6 CCPR1L = 6. DC1B1:DC1B0 = 00 (0)

For 40% Duty cycle Count = 9.6 CCPR1L = 9. DC1B1:DC1B0 = 10 (0.5)

For 50% Duty cycle Count = 12 CCPR1L = 12. DC1B1:DC1B0 = 10 (0)

For 60% Duty cycle Count = 14.4 CCPR1L = 14. DC1B1:DC1B0 = 10 (0.5)

For 75% Duty cycle Count = 18 CCPR1L = 18. DC1B1:DC1B0 = 00 (0)

40% and 60% Duty is not achievable as 0.6 and 0.4 cannot be set by 2-bit decimal value.

Configuring Setup steps for PWM Operation

1. Select the timer (TMR2 or TM4) source for CCPx module. Required in PIC18F8720, not required for PIC18F458 or PIC18C601/801.
2. Set the PWM period by writing to the PR2 (or PR4) register.
3. Set the PWM duty cycle by writing to the CCPRxL register and
4. Select the required decimal part of duty cycle through CCPxCON<5:4> bits.
5. Make the CCPx pin an output by clearing the TRISC<2> bit (For CCP1). Clear the respective TRIS bit for the other CCP's.
6. Set the prescale value by writing to T2CON (or T4CON)
7. Clear TMR2 (or TM4) register. May be set to a value if required.
8. Configure the CCPx module for PWM operation.
9. Start/Enable Timer2 by setting TMR2ON bit – TCON<2> (or Timer 4).

Program 13.8: Generating PWM waveform with 25% duty cycle for a 10Mhz clock using Timer 2 as source.

	ORG	0x0000	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start of Main program.
	ORG	0x0030	; Set Origin 0x0030 for Main program
START	MOVLW	0x00	; WREG = 0x00
	MOVWF	T3CON	; Select Timer 2 for CCP1 by T3CON source select ; Required only in PIC18F8720, not for 458/801
	MOVWF	CCP1CON	; CCP1CON = 0x00, CCP1 module off.
	MOVLW	D'100'	; WREG = 0x64 = 100d
	MOVWF	PR2	; PR2 = 100,
	MOVLW	0X19	; WREG = 0x19 = 25d
	MOVWF	CCPR1L	; CCPR1L = 0x19, with 00 Decimal
	BCF	TRISC,2	; Make CCP1/RC2 pin as output to get CCP ; Square waveform
	CLRF	T2CON	; T2CON = 0X00, Clear Timer 2 with prescale 1
	CLRF	TMR2	; TMR2 = 0x00
	MOVLW	0X0C	; WREG = 0x0C
	IORWF	CCP1CON,1,0	; CCP1CON = CCP1CON OR (WREG = 0x0C), ; Setting CCP1 for PWM
	BSF	T2CON,2	; Start Timer 2
UP	BCF	PIR1, TMR2IF	; Clear TMR2IF flag
L1	BTFSS	PIR1, TMR2IF	; Wait for flag. Check TMR2IF flag. Skip next ; instruction if flag set
	BRA	L1	; Branch to L1, when flag not set

GOTO	UP	; Goto UP, Repeat this process continuously.
END		

Program 13.9: Generating PWM waveform with frequency of 4.9603 KHz with 25% duty cycle with 10 msec off time for a 10Mhz clock using Timer 2 as source.

Sol: From Eq 13.2a & 13.2c

For N= 4 $PR2 = [10MHz / \{4 \times 4.9603 \text{ KHz} \times 4\}] - 1 = 126 - 1 = 125.$
 $TPWM1 = [125+1] \times 4 \times 0.1 \times 10^{-6} \times 4 = 126 \times 1.6 \times 10^{-6} = 201.6 \mu\text{Sec}$
 $fPWM1 = 1/TPWM1 = 4.9603 \text{ KHz}.$
 $PR2 = 125;$
For 25% Duty cycle Count = 31.25 CCPR1L = 31. DC1B1:DC1B0 = 01 (0.25)
For 40% Duty cycle Count = 50 CCPR1L = 50. DC1B1:DC1B0 = 00 (0)
For 50% Duty cycle Count = 62.5 CCPR1L = 62. DC1B1:DC1B0 = 10 (0.5)
For 60% Duty cycle Count = 75 CCPR1L = 75. DC1B1:DC1B0 = 00 (0)
For 75% Duty cycle Count = 93.75 CCPR1L = 93. DC1B1:DC1B0 = 11 (0.75)

	ORG	0x0000	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start of Main program.
	ORG	0x0030	; Set Origin 0x0030 for Main program
START	MOVLW	0x00	; WREG = 0x00
	MOVWF	T3CON	; Select Timer 2 for CCP1 using source selection ; through T3CON, Required in PIC18F8720, not ; required for PIC18F458 or PIC18C601/801
	MOVWF	CCP1CON	; CCP1CON = 0x00, CCP1 module off.
	MOVLW	0x7D	; WREG = 0x7D
	MOVWF	PR2	; PR2 = 0x7D, Generate PWM signal with ; frequency 4.9603 KHz
	MOVLW	0x1F	; WREG = 0x1F
	MOVWF	CCPR1L	; CCPR1L = 0x1F, 31 with 01 Decimal for 25 % ; Duty Cycle, 50 for 40%, 62 with 10 decimal for ; 50%, 75 for 60%, 93 with 11 decimal for 75%.
	BCF	CCP1CON,5	; Set decimal .25 for duty cycle by making 5 bit 0
	BSF	CCP1CON,4	; Set decimal .25 for duty cycle by making 4 bit 1
	BCF	TRISC,2	; Make CCP1/RC2 pin as output to get CCP ; Square waveform
	MOVLW	0x01	; WREG = 0x01
	MOVWF	T2CON	; T2CON = 0x01, Set Timer 2 with prescale 4
	CLRF	TMR2	; TMR2 = 0x00
	MOVLW	0x0C	; WREG = 0x0C
	IORWF	CCP1CON,1,0	; CCP1CON = CCP1CON OR (WREG = 0x0C), ; Setting CCP1 for PWM
	BSF	T2CON,2	; Start Timer 2

UP	BCF	PIR1, TMR2IF	; Clear TMR2IF flag
L1	BTFSF	PIR1, TMR2IF	; Wait for flag. Check TMR2IF flag. Skip next ; instruction if flag set
	BRA	L1	; Branch to L1, when flag not set
	GOTO	UP	; Goto UP, Repeat this process continuously.
END			

This PWM signal generated on CCP1/RC2 pin through the program 13.8 or 13.9 may be used to control the speed of the DC motor. It is shown in fig P13.8. Here the enable pin is connected to Vcc. It may be connected and controlled by PIC Microcontroller when it is connected to an I/O Pin. Active high signal may be connected to signal a & low to b.

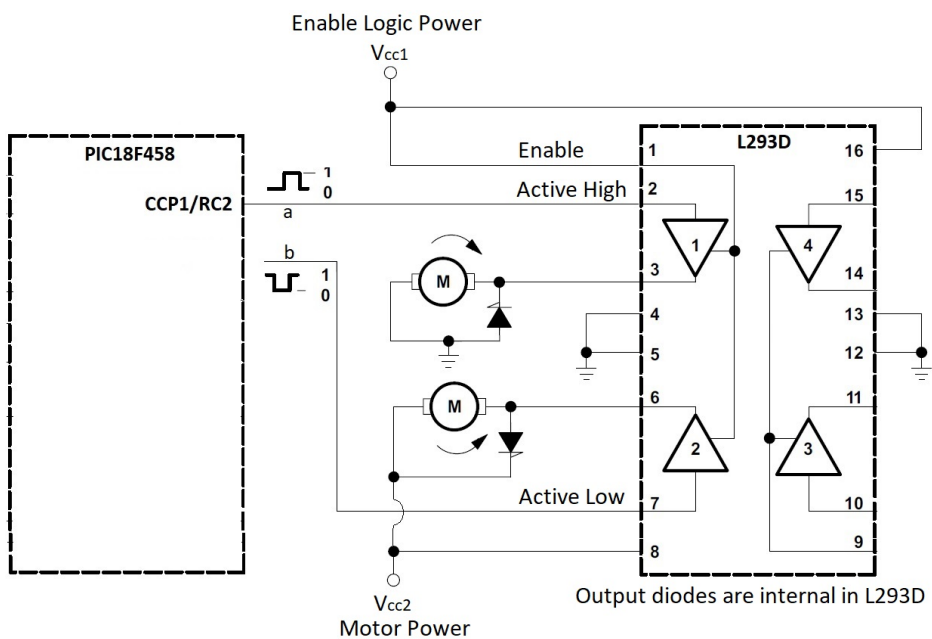


Fig P13.8: Demonstration for program 13.8
Speed control of DC motor using CCP in PWM mode

13.3 Using ECCP

ECCP stands for Enhanced Capture/Compare PWM module. ECCP module is only present in PIC18F458 but is not in PIC18C601/801 or PIC18F8720. ECCP module is similar to the CCP module discussed earlier except for the enhanced PWM. The enhanced PWM allows for up to 4 output channels and features a selectable polarity.

The ECCP1 module uses same structure for capture and compare. So, the timer resource is same as that shown in Table 13.2. The interaction of CCP1 and ECCP1 module is also similar and is as shown in Table 13.4.

The ECCP1 module is similar to that in CCP1 module. The only changes are control register – ECCP1CON (instead of CCP1CON), Holding register ECCPR1H & ECCPR1L (instead of CCPR1H & CCPR1L) and the respective capture input pin ECCP1/RD4 (instead of CCP1/RC2) along with the event flag/enable/priority ECCP1IF/ECCP1IE/ECCP1IP in PIR2/PIE2/IPR2 (instead of CCP1IF in PIR1/PIE1/IPR1). There are additional registers and pins for enhanced PWM functionality. The additional registers are ECCPAS, ECCP1DEL and four additional outputs are present instead of a single output. Except for the PWM delay register ECCP1DEL all the other control registers are double buffered. They are loaded at beginning of new PWM cycle to prevent glitches on any of outputs and due to the buffering, module does not start immediately and waits for the reset of assigned timer. This causes the PWM signals from ECCP1 module to be offset/delayed by a full instruction cycle ($4 T_{OSC}$) w.r.t PWM signals from standard CCP1 module.

Capture Mode:

The capture mode in ECCP1 is similar to CCP1 except for the above-mentioned registers and pin.

Compare Mode:

The capture mode in ECCP1 is identical to CCP1 except for the above-mentioned registers and pin.

The special event trigger output of ECCP1 is identical to that of CCP1 module and can start A/D conversion if A/D module is enabled. The special event trigger output does not set the timer1 and timer3 flags.

PWM Mode:

The ECCP module in PIC18F458 generates PWM signals by working in one of the two operating modes. The duty cycle in PWM mode is set through ECCPR1L register and the two decimal bits through EDC1B1:EDC1B0 of ECCP1CON<5:4>.

ECCCP1CON: Enhanced CCP 1 Control Register Format

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
EPWM1M1	EPWM1M0	EDC1B1	EDC1B0	ECCP1M3	ECCP1M2	ECCP1M1	ECCP1M0
bit 7							bit 0

Bit No	Bit	Description
bit 7-6	EPWM1M<1:0>	PWM Output Configuration bits If ECCP1M<3:2> = 00, 01, 10: xx = P1A assigned as Capture/Compare input; P1B, P1C, P1D assigned as port pins If ECCP1M<3:2> = 11: 00 = Single output; P1A modulated; P1B, P1C, P1D assigned as port pins 01 = Full-bridge output forward; P1D modulated; P1A active; P1B, P1C inactive 10 = Half-bridge output; P1A, P1B modulated with deadband control; P1C, P1D assigned as port pins 11 = Full-bridge output reverse; P1B modulated; P1C active; P1A, P1D inactive
bit 5-4	EDC1B<1:0>	PWM Duty Cycle Least Significant bits Capture mode: Unused. Compare mode: Unused. PWM mode: These bits are the two LSbs of the PWM duty cycle. The eight MSbs are found in ECCPR1L.
bit 3-0	ECCP1M<3:0>	ECCP1 Mode Select bits 0000 = Capture/Compare/PWM off (resets ECCP module) 0001 = Unused (reserved) 0010 = Compare mode, toggle output on match (ECCP1IF bit is set) 0011 = Unused (reserved) 0100 = Capture mode, every falling edge 0101 = Capture mode, every rising edge 0110 = Capture mode, every 4th rising edge 0111 = Capture mode, every 16th rising edge 1000 = Compare mode, set output on match (ECCP1IF bit is set) 1001 = Compare mode, clear output on match (ECCP1IF bit is set) 1010 = Compare mode, ECCP1 pin is unaffected (ECCP1IF bit is set) 1011 = Compare mode, trigger special event (ECCP1IF bit is set; ECCP resets TMR1or TMR3 and starts A/D conversion if A/D module is enabled) 1100 = PWM mode; P1A, P1C active-high; P1B, P1D active-high 1101 = PWM mode; P1A, P1C active-high; P1B, P1D active-low 1110 = PWM mode; P1A, P1C active-low; P1B, P1D active-high 1111 = PWM mode; P1A, P1C active-low; P1B, P1D active-low

- Standard PWM Mode
 - In standard mode the ECCP operates similar to CCP module except the change in ECCP pin, registers, and PWM duty cycle decimal bits in ECCP1CON<5:4>.
- Enhanced PWM Mode

In enhanced mode the ECCP provides multiple configurable PWM output options for varied applications. It is also compatible with standard PWM.

The enhanced features of ECCP are as follows.

- Single, Half or Full-Bridge with Forward/reverse output mode
- 4 output pins (P1A, P1B, P1C & P1D) on RD4:RD7 pins.
- Output Polarity Selection
- Direction Change
- Programmable Dead Band Delay
- Enhanced CCP Auto-Shutdown

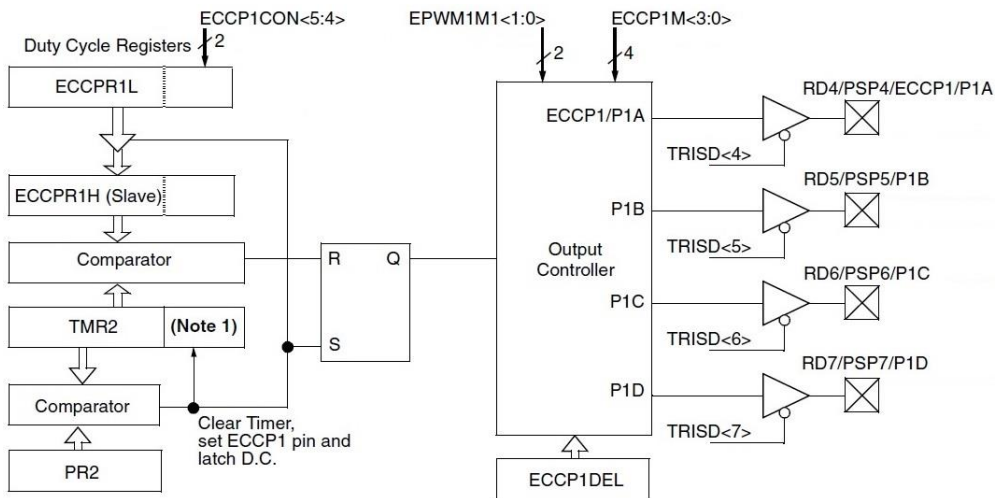


Fig.13.6 Block Diagram of ECCP's PWM Mode in PIC18F458

The block diagram of ECCP's PWM Mode of PIC18F458 is shown in fig.13.6. The EPWM1M1:EPWM1M0 and ECCP1M3:ECCP1M0 bits of the ECCP1CON register (ECCP1CON<7:6> and ECCP1CON<3:0>, respectively) are used to configure the module's output mode, pin assignments and polarity. The auto shut down feature is configured through the ECCPAS register and programmable dead band delay through the ECCP1DEL register. The Timer 4 is present only in PIC18F8720 which does not contain ECCP whereas PIC18F458 contains ECCP but not Timer 4. The registers related to ECCP are presented here. When the ECCP module is PWM mode the PWM pins need to be pulled up/down as the pins are tri-stated (in high impedance state) when the microcontroller is power up.

The specific pin assignments for the various output modes are shown in Table 13-7.

Table 13.7: ECCP Pins on PIC18F458

ECCP Mode	RD4	RD5	RD6	RD7	ECCP1CON Configuration X – Don't Care
Conventional CCP Compatible	ECCP1	RD<5> PSP<5>	RD<6> PSP<6>	RD<7> PSP<7>	00xx11xx
Dual Output PWM	P1A	P1B	RD<6> PSP<6>	RD<7> PSP<7>	10xx11xx
Quad Output PWM	P1A	P1B	P1C	P1D	x1xx11xx

Registers related to Capture, Compare, PWM, Timer1, Timer 2 and Timer3

Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
ECCPR1L	Capture/Compare/PWM Register1 (LSB)							
ECCPR1H	Capture/Compare/PWM Register1 (MSB)							
ECCP1CON	EPWM1 M1	EPWM1 M0	EDC1B1	EDC1B0	ECCP1 M3	ECCP1 M2	ECCP1 M1	ECCP1 M0
ECCPAS	ECCP ASE	ECCP AS2	ECCP AS1	ECCP AS0	PSSAC1	PSSAC0	PSSB D1	PSSB D0
ECCP1DEL	EPDC7	EPDC6	EPDC5	EPDC4	EPDC3	EPDC2	EPDC1	EPDC0
TRISD	PORTD Data Direction Register							
TMR1L	Holding Register for the Least Significant Byte of the 16-bit TMR1 Register							
TMR1H	Holding Register for the Most Significant Byte of the 16-bit TMR1 Register							
T1CON	RD16	—	T1CK PS1	T1CK PS0	T1OSCEN	T1SYNC	TMR1CS	TMR1 ON
TMR3L	Holding Register for the Least Significant Byte of the 16-bit TMR3 Register							
TMR3H	Holding Register for the Most Significant Byte of the 16-bit TMR3 Register							
T3CON	RD16	T3ECCP1	T3CK PS1	T3CK PS0	T3CCP1	T3SYNC	TMR3CS	TMR3 ON
INTCON	GIE/ GIEH	PEIE/ GIEL	TMR0IE	INT0IE	RBIE	TMR0IF	INT0IF	RBIF
PIR2	—	CMIF	—	EEIF	BCLIF	LVDIF	TMR3IF	ECCP1 IF
PIE2	—	CMIE	—	EEIE	BCLIE	LVDIE	TMR3IE	ECCP1 IE
IPR2	—	CMIP	—	EEIP	BCLIP	LVDIP	TMR3IP	ECCP1 IP
RCON	IPEN	—	—	RI	TO	PD	POR	BOR
PWM Specific Registers								
Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
TMR2	Timer2 Module Register							
PR2	Timer2 Module Period Register							
T2CON	—	T2OUT PS3	T2OUT PS2	T2OUT PS1	T2OUT PS0	TMR2 ON	T2CK PS1	T2CK PS0

The output pin polarity (High/Low) and PWM mode selection is done through the mode select bits ECCP1M3:ECCP1M0 (ECCP1CON<3:0>). The PWM output modes are

configured by setting PWM output configuration bits, EPWM1M<1:0> i.e (ECCP1CON<7:6>). The possible output configuration modes are:

- Single Output (Standard PWM Mode) → EPWM1M<1:0> = 00
- Half-Bridge Output (For L293D Half Bridge) → EPWM1M<1:0> = 10
- Full-Bridge Output, Forward mode (For L298 Full Bridge) → EPWM1M<1:0> = 01
- Full-Bridge Output, Reverse mode (For L298 Full Bridge) → EPWM1M<1:0> = 11

Single Output Mode:

This mode is similar to standard PWM mode and a sample output waveform is presented in Fig 13.7. In this mode RD4/ECCP1 need to be set as output by clearing TRISD<4> bit. The other 3 pins RD5, RD6 and RD7 could be used as I/O port pins.

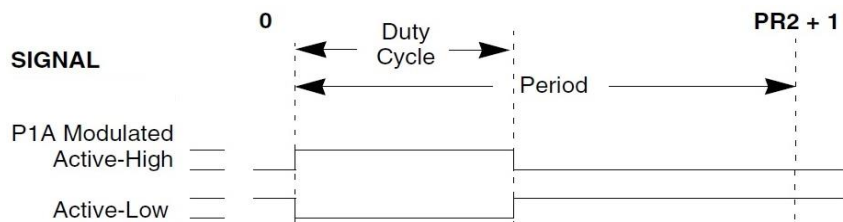


Fig.13.7 Sample Waveform of Single Output – Standard PWM Mode

Half Bridge Mode:

The half bridge mode utilizes two pins P1A (RD4) & P1B (RD5) as the outputs to drive the push pull loads. The PWM output signal is generated on P1A and the complementary signal on P1B. The respective port PWM pins need to be configured as outputs by clearing TRISD<4:5> bits. The other 2 pins RD6 and RD7 could be used as I/O port pins. Sample waveform of Half Bridge PWM output mode is shown in fig 13.8a.

ECCP1DEL: PWM Delay Register Format

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
EPDC7	EPDC6	EPDC5	EPDC4	EPDC3	EPDC2	EPDC1	EPDC0
bit 7				bit 0			

Bit No	Bit	Description
bit 7-0	EPDC<7:0>	PWM Delay Count for Half-Bridge Output Mode bits Number of FOSC/4 (TOSC * 4) cycles between the P1A transition and the P1B transition.

Dead Band Delay

In bridge devices during switching, a shoot through current occurs when both the transistors are on momentarily i.e. when at least one of the pull-up or pull-down transistors is not off. This occurs due to characteristic difference in pull-up/pull-down transistors

The ECCP1DEL register could be programmed to set the number cycles before an output is made active. This is similar to CCPR1L, which when greater than the PR2 register gives



The ECCP1DEL register could be programmed to set the number cycles before an output is made active. This is similar to CCPR1L, which when greater than the PR2 register gives

100% duty cycle. Here the output is never enabled (remains inactive like a 0% duty cycle) if ECCP1DEL register value is greater than the duty cycle. A waveform to demonstrate the dead band delay is shown in fig 13.8b.

A simple push-pull application of ECCP in Half bridge output mode is shown in fig 13.9. In a motor control application, the half bridge signals are typically generated through software program and connected to L293D driver.

Full Bridge Mode:

The full bridge mode utilizes four pins P1A (RD4), P1B (RD5), P1C (RD6) & P1D (RD7) as the outputs. Only two of the four outputs are active at any given time based on the direction – Forward/Reverse. Sample waveform of Full Bridge forward PWM output mode is shown in fig 13.10a and reverse PWM output mode is shown in fig 13.10b. The forward/reverse direction is controlled through the EPWM1M1 bit in ECCP1CON register.

In the forward mode, the two pins RD4/P1A and RD7/P1D are used. The RD4/P1A is continuously active and RD7/P1D acts as the modulated signal. In the reverse mode, the two pins RD5/P1B and RD6/P1C are used. The RD6/P1C is continuously active and RD5/P1B acts as the modulated signal. The respective port pins need to be configured as outputs by clearing TRISD<4:7> bits.

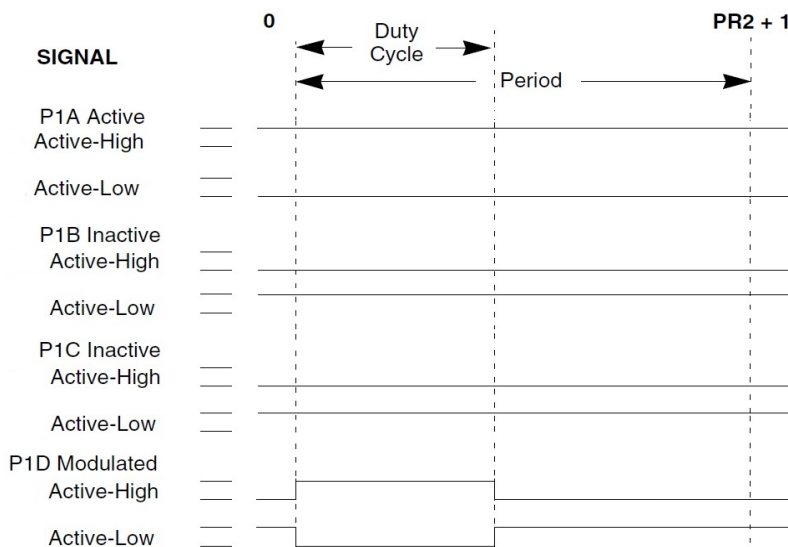


Fig.13.10a) Sample Waveform of Full Bridge Output – Forward Mode

A simple application of ECCP in Full bridge output mode is shown in fig 13.11. In a motor control application, the full bridge signals are typically generated through software program and connected to L298 driver. Two interfacing case studies full bridge are

presented. The example program 13.13 demonstrates the DC motor control using full bridge ECCP1 PWM forward output mode. Another equivalent example is demonstrated in interfacing and speed control of DC motors section without the ECCP module.

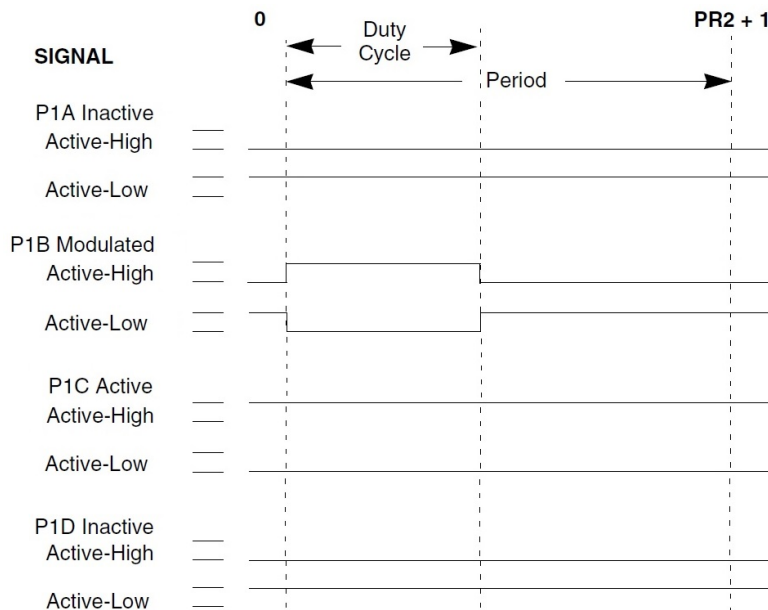


Fig.13.10b) Sample Waveform of Full Bridge Output – Reverse Mode

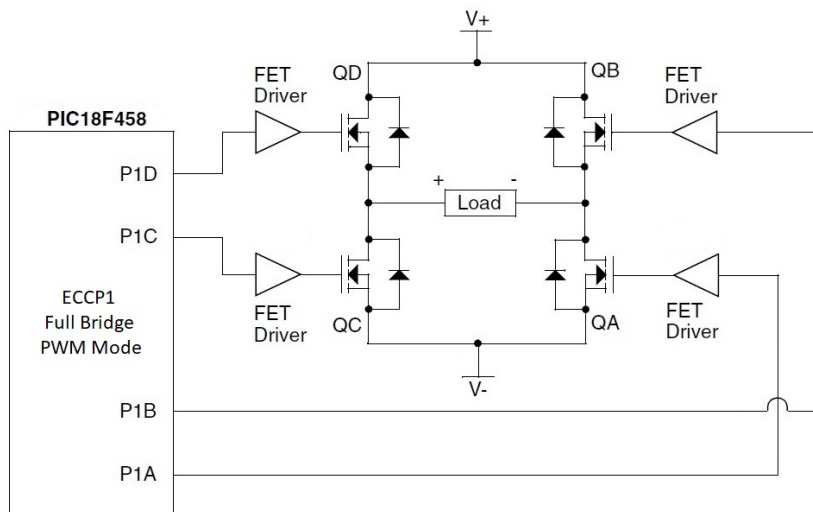


Fig.13.11 Full Bridge Output mode (Push-Pull) application

Note:

- **Polarities:** PWM output polarities need to set through ECCP1M<1:0> bits before PWM outputs are enabled. It is recommended that change in polarity is done when the PWM pins are not configured as outputs for predictable operations.
- **PWM Pin:** To ensure the proper latch state and to protect power switch devices, ECCP1 module must be enabled with proper output mode first with PWM pins P1A, P1B, P1C and P1D pins configured as inputs using TRISD bits. After a full PWM cycle the latches are then properly initialized to proper state following which the PWM pins P1A, P1B, P1C and P1D pins may be configured as outputs to drive the power switch devices.
- **Direction Change:** Proper care needs to be taken while switching the directions in full bridge mode. When direction control bit is changed in the firmware the current PWM cycle continues in the initial direction and the new direction is applied only in the next PWM cycle/period. When the signals are connected to Motors, the motor need to be allowed sufficient time for changing the direction.
- **Dead-band delay:** Although dead-band delay is generally not required in full bridge as only one output is modulated. In case of direction changes etc it may still be required.

Enhanced CCP Auto-Shutdown

The ECCP module has another additional feature compared to a traditional CCP module. The ECCP module could be configured to auto-shutdown based on associated internal output from a different module or from an external interrupt 0. Analog or a digital event could be used to discontinue the ECCP sequence. To auto-shutdown, the respective configured modules need to be enabled and INTOIE must be set for the external interrupt INTO. The ECCP module could be programmed to auto-shutdown for the following events.

- Comparator 1
- Comparator 2
- INTO

The configuration for auto-shutdown could be performed through the ECCPAS register. The format of the register is given below. The values of (PSSACn, PSSBDn) bits in ECCPAS<3:0> register is written to ECCP pins when an auto-shutdown occurs. The internal shutdown event is internally gated and disables the PWM outputs asynchronously. The ECCPASE bit in ECCPAS <7> register is status flag bit. It is set by internal hardware when a shutdown is caused by comparator event and need to be cleared in software for re-enabling the ECCP outputs. The ECCPASE bit can be set through software program to create a manual shutdown.

ECCPAS: Enhanced Capture/Compare/PWM Auto-Shutdown Control Register Format

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
ECCPASE	ECCPAS2	ECCPAS1	ECCPAS0	PSSAC1	PSSAC0	PSSBD1	PSSBD0
bit 7				bit 0			
Bit No	Bit	Description					
bit 7	ECCPASE	ECCP Auto-Shutdown Event Status bit 0 = ECCP outputs enabled, no shutdown event 1 = A shutdown event has occurred, must be reset in software to re-enable ECCP					
bit 6-4	ECCPAS<2:0>	ECCP Auto-Shutdown bits 000 = No auto-shutdown enabled, comparators have no effect on ECCP 001 = Comparator 1 output will cause shutdown 010 = Comparator 2 output will cause shutdown 011 = Either Comparator 1 or 2 can cause shutdown 100 = INTO 101 = INTO or Comparator 1 output 110 = INTO or Comparator 2 output 111 = INTO or Comparator 1 or Comparator 2 output					
bit 3-2	PSSACn	Pins A and C Shutdown State Control bits 00 = Drive Pins A and C to '0' 01 = Drive Pins A and C to '1' 1x = Pins A and C tri-state					
bit 1-0	PSSBDn	Pins B and D Shutdown State Control bits 00 = Drive Pins B and D to '0' 01 = Drive Pins B and D to '1' 1x = Pins B and D tri-state					

Configuring Setup steps for PWM Operation using ECCP

1. Make the ECCP's output pins as inputs by setting the respective TRIS<4:7> bits.
 - In single mode only TRIS<4>, Half Bridge mode only TRIS<4:5> else all.
2. Set the PWM period by writing to the PR2 register.
3. Set the PWM duty cycle by writing to the ECCPRxL register
4. Select the required decimal part of duty cycle through ECCPxCON<5:4> bits.
5. Load the ECCP1CON register (ECCP1M<3:0> and EPWM1M<1:0>bits) with value as per the required mode and polarity to configure the ECCP1 module.
 - If the mode selected is Half bridge, set the dead band delay by writing to ECCP1DEL if required.
6. Clear TMR2 (or TM4) register. May be set to a value if required.
7. Clear TMR2IF interrupt flag of Timer 2 in PIR1 register.
8. Set the prescale value by writing to T2CKPS bits - T2CON<1:0>.

9. Start/Enable Timer2 by setting TMR2ON bit – TCON<2>.
10. Enable PWM outputs after the start of a new cycle. i.e check till TMR2IF is set to 1.
11. Make the ECCP's output pins as outputs by clearing the respective TRIS<4:7> bits.

Program 13.10a): Determining the Time Period/Frequency, Store No of pulses in 0x30,0x31 and send to PortB and PortC. Using ECCP1 and Timer 1 in **capture** mode. Equivalent of Program 13.1

	ORG	0x0000	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start of Main program.
	ORG	0x0030	; Set Origin 0x0030 for Main program
START	CLRF	TRISB	; Make Port B as output
	CLRF	TRISC	; Make Port C as output
	MOVLW	0x05	; WREG = 0x05
	MOVWF	ECCP1CON	; ECCP1CON = WREG = 0x05, Setting CCP1 for ; Capture mode with rising edge with 0 prescale
	BSF	TRISD,4	; Make ECCP1/RD4/P1A pin as input for capture
	MOVLW	0x00	; WREG = 0x00
	MOVWF	T3CON	; T3CON = 0x00, Cleared to Select Timer 1 ; as time source for ECCP
	MOVWF	T1CON	; T1CON = 0x00, Select 8-bit read/write ; operations of Timer1, with Prescale 1, ; select internal main clock fosc/4
UP	CLRF	TMR1H	; Initialize the Timer 1 high TMR1H to 0x00
	CLRF	TMR1L	; Initialize the Timer 1 low TMR1L to 0x00
	BCF	PIR2,ECCP1IF	; Clear ECCP1IF flag
L1	BTFSS	PIR1,ECCP1IF	; Wait for first rising edge event. Check ECCP1IF ; flag. Skip next instruction if event flag
	BRA	L1	; Branch to L1 if event flag not set
	BSF	T1CON,0	; Start the Timer 1 TMR1ON to see number of ; pulses till next event flag.
	BCF	PIR2,ECCP1IF	; Clear ECCP1IF flag
L2	BTFSS	PIR2,ECCP1IF	; Wait for second rising edge event. Check ; ECCP1IF flag. Skip next instruction if event flag
	BRA	L2	; Branch to L2 if event flag not set
	BCF	T1CON,0	; Stop the Timer 1. TMR1ON
	MOVF	ECCPR1H,W	; WREG = ECCPR1H, Copy latched Timer1 High
	MOVWF	0x31	; [0x31] = ECCPR1H = WREG
	MOVF	ECCPR1L,W	; WREG = ECCPR1L, Copy latched Timer1 Low
	MOVWF	0x30	; [0x30] = CCPR1L = WREG
	MOVFF	0x30,PORTB	; PORTB = ECCPR1L = [0x30]
	MOVFF	0x31,PORTC	; PORTC = ECCPR1H = [0x31]

GOTO	UP	; Goto UP, Repeat this process continuously.
END		

Program 13.10b): Program of 13.10a with interrupts Equivalent of Program 13.3

	ORG	0x0000	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start of Main program.
	ORG	0x0008	; Set Origin 0x0008 of High Priority Interrupt ; start address
	GOTO	CCP_ISR	; Go to CCP_ISR
	ORG	0x0030	; Set Origin 0x0030 for CCP_ISR
CCP_ISR	BTFSS	PIR2,ECCP1IF	; Check if the interrupt is due to ECCP1IF. Skip ; next instruction if its due to ECCP1IF event flag.
	BRA	L1	; If not due to ECCP1IF go to L1, do nothing, ; come out of ISR.
	BTFSC	0x00,0	; Check if its first/second ECCP1IF event flag, by ; checking LSB bit of file register 0x00. Skip next ; instruction if clear, i.e. due to first event flag.
	BRA	L2	; Branch to L2 if its due to second event flag.
	BSF	0x00,0	; Set LSB bit of file register 0x00.
	CLRF	TMR1H	; Initialize Timer 1 high TMR1H to 0x00
	CLRF	TMR1L	; Initialize Timer 1 low TMR1L to 0x00
	BSF	T1CON,0	; Start the Timer 1 to see number of pulses till ; next event flag.
	BRA	L3	; Branch to L3 to come to end of ISR
L2	BCF	0x00,0	; Clear LSB bit of file register 0x00.
	CALL	DSEND	; Call DSEND, to get Pulse count, store/send
L3	BCF	PIR2,ECCP1IF	; Clear ECCP1IF flag
L1	NOP		; No operation
	RETFIE		; Return from ISR, Go back to main program
	ORG	0x0100	; Set Origin 0x0100 for module DSEND
DSEND	MOVF	ECCPR1H,W	; WREG = ECCPR1H, Copy latched Timer1 High
	MOVWF	0x31	; [0x31] = ECCPR1H = WREG
	MOVF	ECCPR1L,W	; WREG = ECCPR1L, Copy latched Timer1 Low
	MOVWF	0x30	; [0x30] = ECCPR1L = WREG
	MOVFF	0x30,PORTB	; Send Low count PORTB = ECCPR1L = [0x30]
	MOVFF	0x31,PORTC	; Send High count PORTC = ECCPR1H = [0x31]
	BCF	T1CON, 0	; Stop the Timer 1.
	RET		; Return from module DSEND
	ORG	0x0500	; Set Origin 0x0500 for Main program
START	CLRF	TRISB	; Make Port B as output
	CLRF	TRISC	; Make Port C as output

	MOVLW	0X05	; WREG = 0x05
	MOVWF	ECCP1CON	; ECCP1CON = WREG = 0x05, Setting ECCP1 for ; Capture mode with rising edge with 0 prescale
	BSF	TRISD,4	; Make ECCP1/RD4 pin as input for capture
	MOVLW	0x00	; WREG = 0x00
	MOVWF	T3CON	; T3CON = 0x00, Cleared to Select Timer 1 as ; time source for ECCP
	MOVWF	T1CON	; T1CON =0x00, Select 8-bit read/write ; operations of Timer1, with Prescale 1, select ; internal main clock fosc/4
	BSF	INTON,GIE	; Enable Global interrupt
	BSF	INTON,PEIE	; Enable Peripheral interrupts
	BSF	RCON,IPEN	; Enable interrupt priorities
	BSF	IPR2,ECCP1IP	; Make ECCP1IF as high priority
	BSF	PIE2,ECCP1IE	; Enable ECCP1IF interrupt
	BCF	PIR2,ECCP1IF	; Clear ECCP1IF flag
UP	NOP		; No operation, Logic of optional alternate ; function may be placed here.
	GOTO	UP	; Goto UP, Repeat this process continuously.
	END		

Program 13.11: Generating square wave of 60% duty cycle with 10 msec off time for a 10Mhz clock using ECCP1 in **compare** mode. Equivalent of Program 13.6

Sol: 10msec off time $\rightarrow T_{off} = T * 0.4 = 10\text{msec} \rightarrow \text{Period } T = 10\text{msec}/0.4 = 25\text{msec}$.

So ON time 60% of $T = T_{ON} = 0.6 * 25\text{msec} = 15\text{msec}$.

Clock Period = $T = 1/(f_{osc}/4) = 1/\{(10 \times 10^6)/4\} = 0.4 \mu\text{sec}$.

For ON time of 25 msec $\rightarrow \text{Count value} = 25\text{msec}/0.4 \mu\text{sec} = 62500 = 0xF424$

For ON time of 10 msec $\rightarrow \text{Count value} = 10\text{msec}/0.4 \mu\text{sec} = 25000 = 0x61A8$

	ORG	0x0000	; Set Origin 0x0000 of Flash memory
	GOTO	ST	; Go to Start of Main program.
	ORG	0x0030	; Set Origin 0x0030 for Main program
ST	MOVLW	0X02	; WREG = 0x02
	MOVWF	ECCP1CON	; ECCP1CON = WREG = 0x02, Set ECCP1 for ; Compare, Toggle on match, use 0x08 to initialize ; pin low & set upon match, 0x09 for opposite.
	BCF	TRISD,4	; Make ECCP1/RD4 pin O/P to get ECCP Square WF
	BSF	TRISC,0	; Make T1CLKI/RC0 pin input to get input
	MOVLW	0x00	; WREG = 0x00
	MOVWF	T3CON	; T3CON = 0x00, Cleared to Select Timer 1 as time ; source for ECCP
	MOVLW	0x02	; WREG = 0x02

	MOVWF	T1CON	; T1CON =0x02, Select 8-bit read/write op ; of Timer1, with Prescale 1, select T1CLKI
UP	CLRF	TMR1H	; Initialize Timer 1 high count reg TMR1H to 0x00
	CLRF	TMR1L	; Initialize Timer 1 low count reg TMR1L to 0x00
	BTFSC	0x00,0	; Check if its first/second ECCP1IF event flag, by ; checking LSB bit of file register 0x00. Skip next ; instruction if its clear, i.e. due to 1st event flag.
	BRA	L1	; Branch to L1, load ECCPR1 reg with ON time value
	MOV	ECCPR1H,0x61	; ECCPR1H = 0x61, Load High byte of OFF time
	MOV	ECCPR1L,0xA8	; ECCPR1L = 0xA8, Load Low byte of OFF time
	BRA	L2	; Branch to L2 to continue OFF time part
L1	MOV	ECCPR1H,0xF4	; ECCPR1H = 0xF4, Load High byte of ON time
	MOV	ECCPR1L,0x24	; ECCPR1L = 0x24, Load Low byte of ON time
L2	BTG	0x00,0	; Toggle LSB bit of file register 0x00.
	BCF	PIR2,ECCPIF	; Clear ECCP1IF flag
	BSF	T1CON,0	; Start the Timer 1 to count the time pulses
L1	BTFSS	PIR2,ECCP1IF	; Wait for first rising edge event. Check ECCP1IF ; flag. Skip next instruction if event flag
	BRA	L1	; Branch to L1 if event flag not set
	BCF	T1CON,0	; Stop the Timer 1
	GOTO	UP	; Goto UP, Repeat this process continuously.
	END		

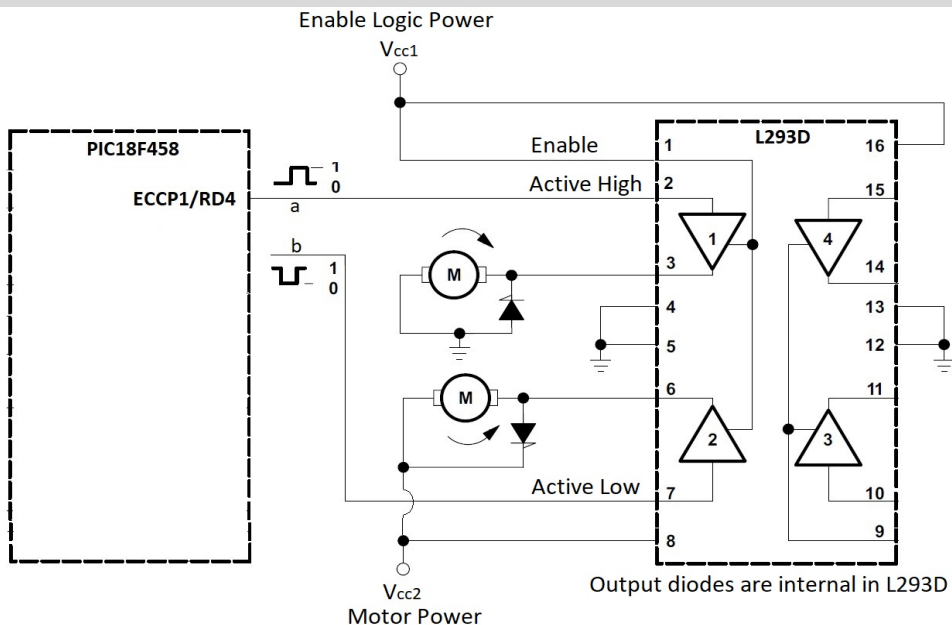


Fig P13.12: Demonstration for program 13.12
Speed control of DC motor using ECCP in PWM Single Output mode

Program 13.12: Generating PWM waveform with frequency of 4.9603 KHz with 25% duty cycle with 10 msec off time for a 10Mhz clock using Timer 2 as source using ECCP1 module.

Sol: From Eq 13.2a & 13.2c

For N= 4 PR2 = $[10\text{MHz}/\{4 \times 4.9603 \text{ KHz} \times 4\}] - 1 = 126 - 1 = 125$.
TPWM1 = $[125+1] \times 4 \times 0.1 \times 10^{-6} \times 4 = 126 \times 1.6 \times 10^{-6} = 201.6 \mu\text{Sec}$
fPWM1 = $1/\text{TPWM1} = 4.9603 \text{ KHz}$.
PR2 = 125;
For 25% Duty cycle Count = 31.25 CCP1L = 31. DC1B1:DC1B0 = 01 (0.25)
For 40% Duty cycle Count = 50 CCP1L = 50. DC1B1:DC1B0 = 00 (0)
For 50% Duty cycle Count = 62.5 CCP1L = 62. DC1B1:DC1B0 = 10 (0.5)
For 60% Duty cycle Count = 75 CCP1L = 75. DC1B1:DC1B0 = 00 (0)
For 75% Duty cycle Count = 93.75 CCP1L = 93. DC1B1:DC1B0 = 11 (0.75)

	ORG	0x0000	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start of Main program.
	ORG	0x0030	; Set Origin 0x0030 for Main program
START	MOVWF	ECCP1CON	; ECCP1CON = 0x00, ECCP1 module off.
	BSF	TRISD,4	; Make ECCP1/RD4 pin as input to disable
			; PWM output
	MOVLW	0x7D	; WREG = 0x7D
	MOVWF	PR2	; PR2 = 0x7D, Generate PWM signal with
			; frequency 4.9603 KHz
	MOVLW	0x1F	; WREG = 0x1F
	MOVWF	ECCPR1L	; ECCPR1L = 0x1F, 31 with 01 Decimal for 25 %
			; Duty Cycle, 50 for 40%, 62 with 10 decimal for
			; 50%, 75 for 60%, 93 with 11 decimal for 75%.
	BCF	ECCP1CON,5	; Set decimal .25 for duty cycle by making 5 bit 0
	BSF	ECCP1CON,4	; Set decimal .25 for duty cycle by making 4 bit 1
	MOVLW	0x0C	; WREG = 0x0C
	IORWF	ECCP1CON,1,0	; ECCP1CON = ECCP1CON OR (WREG = 0x0C),
			; Setting CCP1 for PWM
	CLRF	TMR2	; TMR2 = 0x00
	BCF	PIR1, TMR2IF	; Clear TMR2IF flag
	MOVLW	0x01	; WREG = 0x01
	MOVWF	T2CON	; T2CON = 0x01, Set Timer 2 with prescale 4
	BSF	T2CON,2	; Start Timer 2
L1	BTFSS	PIR1, TMR2IF	; Wait for flag. Check TMR2IF flag. Skip next
			; instruction if flag set
	BRA	L1	; Branch to L1, when flag not set
	BCF	TRISD,4	; Make ECCP1/RD4 pin as output to get ECCP1
			; Square waveform
UP	BCF	PIR1, TMR2IF	; Clear TMR2IF flag

L2	BTFSS	PIR1, TMR2IF	; Wait for flag. Check TMR2IF flag. Skip next ; instruction if flag set
	BRA	L2	; Branch to L2, when flag not set
	GOTO	UP	; Goto UP, Repeat this process continuously.
	END		

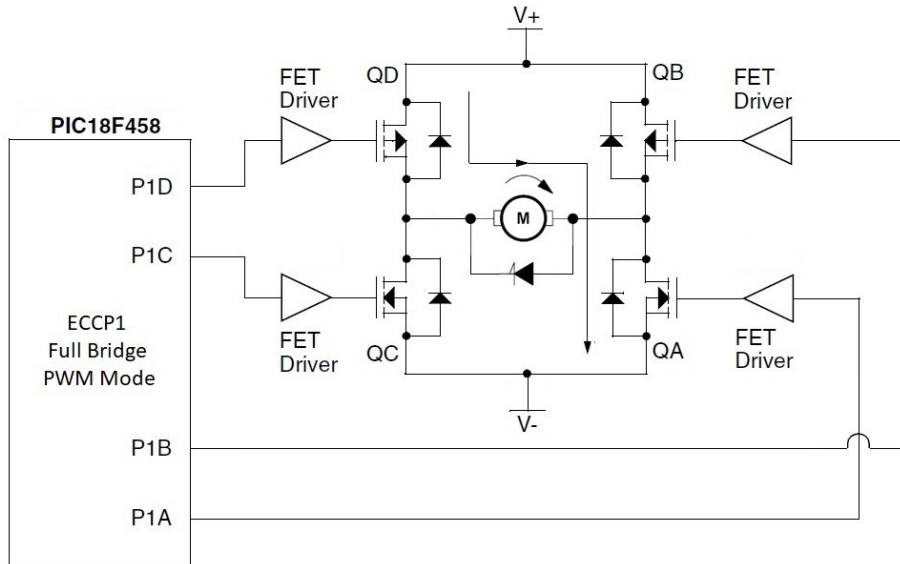
This PWM signal generated on ECCP1/RD4 pin through the program 13.12 may be used to control the speed of the DC motor. It is shown in fig P13.12. Here the enable pin is connected to Vcc. It may actually be connected and controlled by PIC Microcontroller when it is connected to an I/O Pin. Active high signal may be connected to signal a and low to b.

Program 13.13: Generating PWM waveform with frequency of 4.9603 KHz with 25% duty cycle with 10 msec off time for a 10Mhz clock using Timer 2 as source using ECCP1 module using full bridge output forward mode.

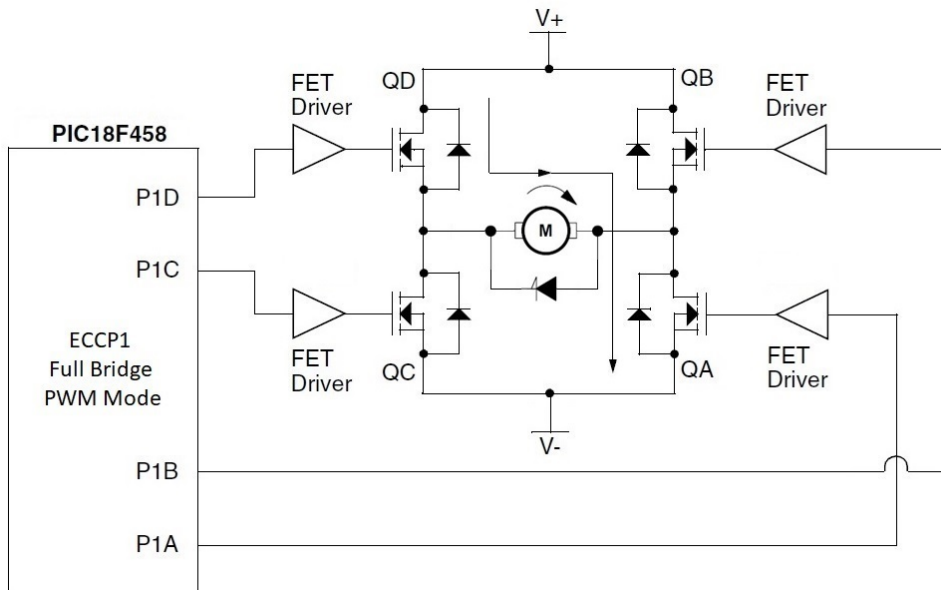
A configuration shown in fig P13.13a requires P1D to drive the QD P-type pull up transistor to turn ON and QA N-type pull down transistor. This requires P1D to be active low and P1A to be active high. This will turn the motor in forward direction. Similarly, fig P13.13b requires P1D to be active high for N-Pull up and P1A to be active high. In fig P13.13c when P1B is active low for a QB P-pull up and P1C connected to QC N-pull down is active high the motor turns in reverse direction. Similarly, fig P13.13d requires P1B to be active high for N-Pull up and P1C to be active high. The same is summarized in table P13.13.

Table P13.3

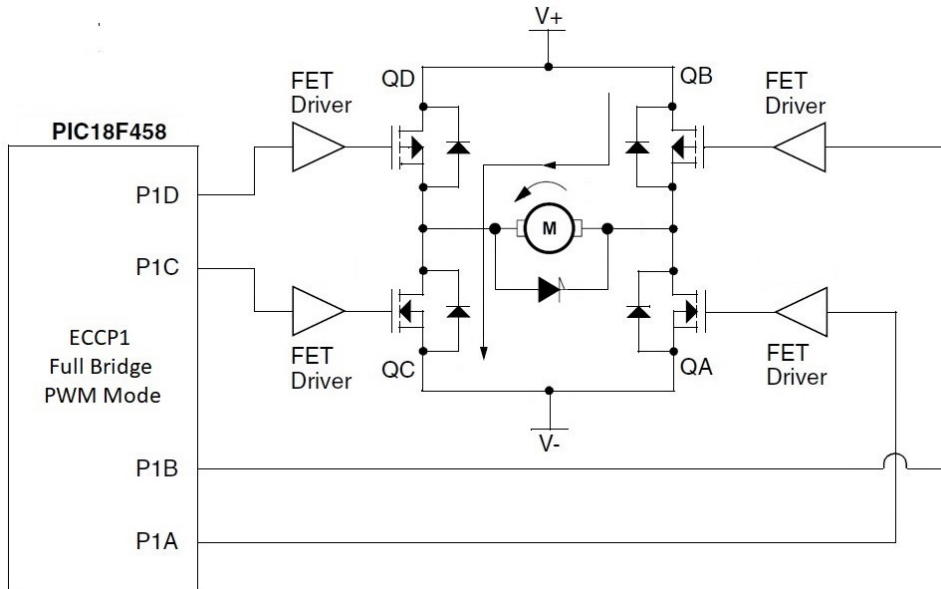
Output Mode/Pin		QA/ P1A	QB/ P1B	QC/ P1C	QD/ P1D	ECCP1CON Value/ Decimal-0.25	
Forward	P-Pull UP	1	0-x	1-x	0	01xx 1101	0x5D
	N-Pull UP	1	1-x	1-x	1	01xx 1100	0x5C
Reverse	P-Pull UP	1-x	0	1	0-x	11xx 1101	0xCD
	N-Pull UP	1-x	1	1	1-x	11xx 1100	0xCC



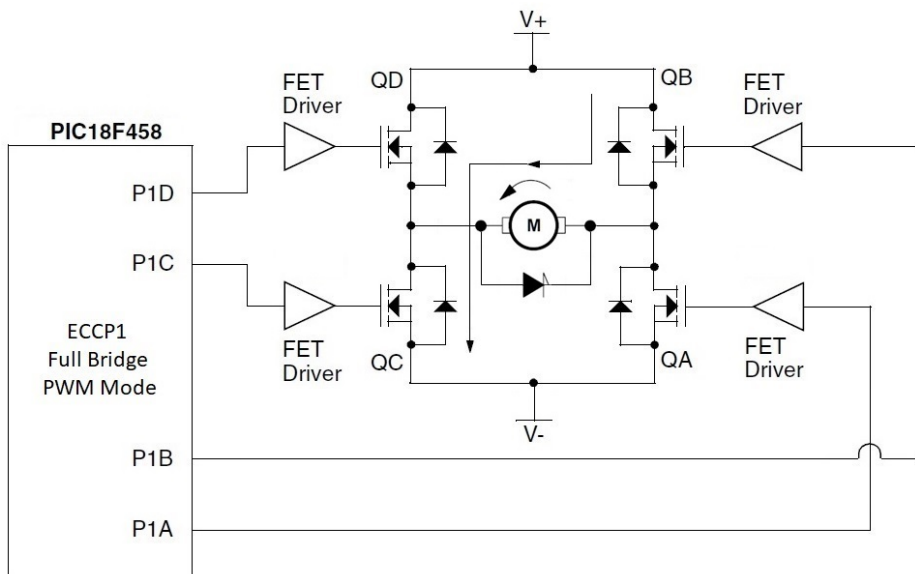
**Fig P13.13a): Demonstration for program 13.13 - Speed control of DC motor
Using ECCP in PWM Full Bridge Forward Output mode Active Low
Require ECCP1CON value of 01xx 1101 i.e 0x5D**



**Fig P13.13b): Demonstration for program 13.13 - Speed control of DC motor
Using ECCP in PWM Full Bridge Forward Output mode Active High
Require ECCP1CON value of 01xx 1100 i.e 0x5C**



**Fig P13.13c): Demonstration for program 13.13 - Speed control of DC motor
Using ECCP in PWM Full Bridge Reverse Output mode Active Low
Require ECCP1CON value of 11xx 1101 i.e 0xCD**



**Fig P13.13d): Demonstration for program 13.13 - Speed control of DC motor
Using ECCP in PWM Full Bridge Reverse Output mode Active High
Require ECCP1CON value of 11xx 1100 i.e 0xCC**

Table P13.13: PWM decimal points

	ORG	0x0000	; Set Origin 0x0000 of Flash memory
	GOTO	START	; Go to Start of Main program.
	ORG	0x0030	; Set Origin 0x0030 for Main program
START	MOVWF	ECCP1CON	; ECCP1CON = 0x00, ECCP1 module off.
	CLRF	TRISD	; Clear TRISD, PortD as I/P to disable PWM O/P
	MOVLW	0x7D	; WREG = 0x7D
	MOVWF	PR2	; PR2 = 0x7D, PWM frequency 4.9603 KHz
	MOVLW	0x1F	; WREG = 0x1F
	MOVWF	ECCPR1L	; ECCPR1L = 0x1F, 31 with 01 Decimal for 25 %
	BCF	ECCP1CON,5	; Set decimal .25 for duty cycle by making 5 bit 0
	BSF	ECCP1CON,4	; Set decimal .25 for duty cycle by making 4 bit 1
	MOVLW	0x4D	; WREG = 0x4D, Setting CCP1 for PWM
	IORWF	ECCP1CON,1,0	; ECCP1CON = ECCP1CON OR WREG = 0x5D ; ECCP1CON = (0x10 OR 0x4D) = 0x5D
	CLRF	TMR2	; TMR2 = 0x00
	BCF	PIR1, TMR2IF	; Clear TMR2IF flag
	MOVLW	0x01	; WREG = 0x01
	MOVWF	T2CON	; T2CON = 0x01, Set Timer 2 with prescale 4
	BSF	T2CON,2	; Start Timer 2
L1	BTFSS	PIR1, TMR2IF	; Wait for flag. Check TMR2IF flag. Skip next ; instruction if flag set
	BRA	L1	; Branch to L1, when flag not set
	SETF	TRISD	; Set TRISD, PortD as PWM O/P to get ECCP1 ; Square waveform
UP	BCF	PIR1, TMR2IF	; Clear TMR2IF flag
L2	BTFSS	PIR1, TMR2IF	; Wait for flag. Check TMR2IF flag. Skip next ; instruction if flag set
	BRA	L2	; Branch to L2, when flag not set
	GOTO	UP	; Goto UP, Repeat this process continuously.
	END		